

AppStore

관리자 가이드

버전	v2.11.4
작성일	2026-09-21
문서	관리자 가이드

AppStore 관리자 가이드

대상 버전: v2.11.4 · 실린 화면은 모두 이 버전을 실제로 띄워 찍은 것입니다.

화면을 쓰는 사람을 위한 설명은 [사용자 가이드](#)에 있습니다. 이 문서는 그 화면을 띄워 놓고 지키는 사람을 위한 것입니다.

1. 구성 요소

AppStore는 컨테이너 하나로 동작합니다. React SPA, REST API, OIDC 세션, AI SSE 프록시, MCP endpoint, PostgreSQL 마이그레이션과 초기 카탈로그가 모두 하나의 Go 바이너리 안에 들어 있습니다.

구성 요소	필수	역할	비고
appstore:vX.Y.Z 컨테이너	예	SPA + API + MCP + AI 프록시	Alpine 기반, non-root appstore:appstore (uid/gid 10001), read-only rootfs
PostgreSQL	예	유일한 영속 저장소	앱·사용자·역할·설정·암호화된 secret·감사 로그 전부
Keycloak (OIDC)	아니오	조직 SSO	없으면 Bootstrap 관리자만 로그인 가능
AI Provider	아니오	AI 스트리밍 응답	OpenAI 호환 / vLLM / Ollama / Custom
Reverse proxy	권장	HTTPS 종단, 외부 노출	컨테이너는 평문 8080만 listen

주고받는 것:

방향	상대	내용
AppStore → PostgreSQL	TCP, DSN의 sslmode 설정에 따름	마이그레이션(시작 시 advisory lock 아래 자동), 조회·변경
브라우저 → AppStore	HTTP 8080	SPA, /api/v1, /mcp, /openapi.json, /docs
AppStore → Keycloak	HTTPS	discovery 문서, token, userinfo, JWKS
AppStore → AI Provider	HTTP(S)	채팅 스트리밍 프록시
AppStore → 임의 이미지 주소	HTTP(S)	로고·파비콘 주소에서 가져오기를 쓸 때만

컨테이너에 영속 볼륨이 없습니다. 루트 파일 시스템은 읽기 전용이고 /tmp 만 64MB tmpfs입니다.

2. 설치

릴리스 자산 하나(`appstore-vX.Y.Z.tar.gz`)로 펌웨어까지 반입할 수 있습니다. 아래는 v2.11.4 기준입니다.

2.1. 선행 조건

항목	요구
호스트	Linux/AMD64 + Docker Engine
PostgreSQL	스키마 변경 권한이 있는 데이터베이스. 마이그레이션이 확장 설치 없이 <code>gen_random_uuid()</code> 를 쓰므로 PostgreSQL 13 이상
네트워크	컨테이너에서 PostgreSQL을 이름으로 찾을 수 있어야 합니다
포트	컨테이너 8080/tcp 하나. 볼륨 없음
자원	서비스 자체는 가볍습니다. 부하 대부분은 PostgreSQL에 걸립니다
SSO를 쓸 경우	도달 가능한 Keycloak Issuer와 신뢰되는 TLS 체인

PostgreSQL 이미지는 릴리스에 포함되지 않습니다. 먼저 준비하세요.

2.2. 이미지 반입과 검증

```
sha256sum appstore-v2.11.4.tar.gz          # 릴리스 노트의 SHA-256과 대조
gzip -t appstore-v2.11.4.tar.gz
gzip -dc appstore-v2.11.4.tar.gz | docker load
docker image inspect appstore:v2.11.4 \
  --format '{{.RepoTags}} user={{.Config.User}} version={{index .Config.Labels "org.opencontainers.image.ve
```

`user=appstore:appstore` 와 archive 이름에 맞는 version label이 보여야 합니다. 하나라도 다르면 배포하지 마세요.

소스에서 직접 빌드할 수도 있습니다.

```
docker build \
  --build-arg VERSION=v2.11.4 \
  --build-arg COMMIT="$(git rev-parse HEAD)" \
  --build-arg BUILD_DATE="$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  -t appstore:v2.11.4 .
```

2.3. 환경 파일 준비

```
install -m 600 .env.example appstore.env
openssl rand -base64 32 # ENCRYPTION_KEY 값 생성
```

```
# appstore.env
POSTGRES_DSN=postgres://appstore:ENCODED_PASSWORD@postgres.example.internal:5432/appstore?sslmode=require
BOOTSTRAP_ADMIN=admin
BOOTSTRAP_ADMIN_PASSWORD=replace-with-a-long-random-password
ENCRYPTION_KEY=replace-with-the-generated-32-byte-base64-value
```

값은 모두 예시입니다. 실제 비밀값을 저장소나 이미지에 넣지 마세요.

2.4. 실행

docker run :

```
docker run -d \
  --name appstore \
  --restart unless-stopped \
  --read-only \
  --tmpfs /tmp:size=64m,mode=1777 \
  --cap-drop ALL \
  --security-opt no-new-privileges:true \
  --env-file ./appstore.env \
  -p 127.0.0.1:8080:8080 \
  appstore:v2.11.4
```

Docker Compose (저장소의 `docker-compose.yml`):

```
APPSTORE_VERSION=v2.11.4 docker compose up -d --no-build
docker compose ps
docker compose logs --tail 100 appstore
```

`APPSTORE_VERSION` · `APPSTORE_COMMIT` · `APPSTORE_BUILD_DATE` 는 Compose가 이미지 태그와 라벨을 고르는 호스트 쪽 치환 값이며 컨테이너 안으로 전달되지 않습니다.

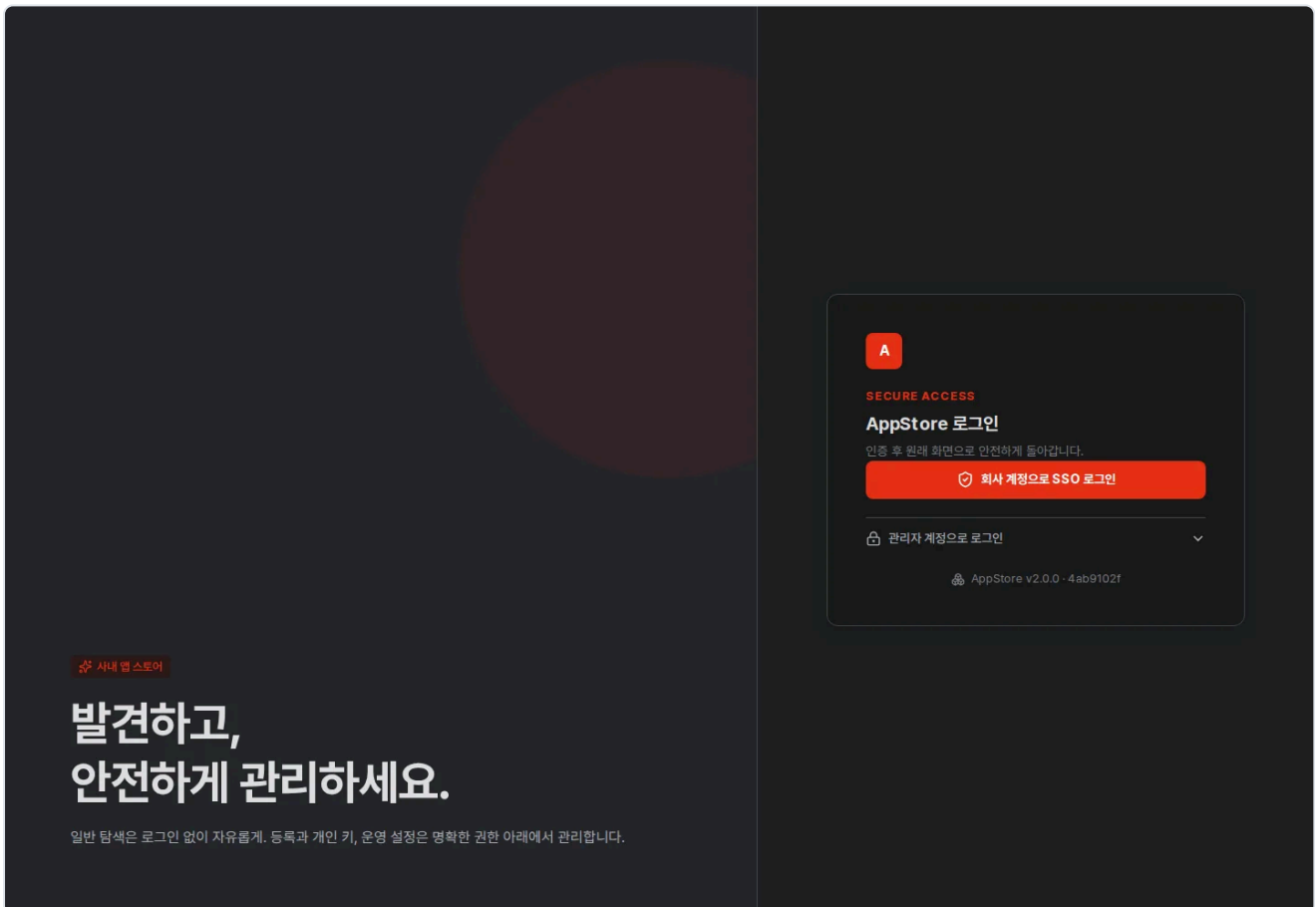
2.5. 기동 확인

```
curl --fail http://127.0.0.1:8080/health/live      # 프로세스 응답
curl --fail http://127.0.0.1:8080/health/ready    # DB 연결과 마이그레이션 완료
curl --fail http://127.0.0.1:8080/api/version     # version / commit / buildDate
```

첫 기동에서 서비스는 순서대로 마이그레이션 적용 → ENCRYPTION_KEY 지문 확인 → 초기 카탈로그 seed → Bootstrap 관리자 생성 → 내장 가이드 첨부을 수행합니다. /health/ready 가 200을 돌려주면 여기까지 끝난 상태입니다. 마지막 단계는 로그에 bundled guides synced 로 남고 apps (짜이 맞은 앱 수) · attached · replaced · unchanged · unmatched (같은 slug의 앱이 아직 없는 가이드) · failed 수를 함께 적습니다.

2.6. 최초 관리자 계정

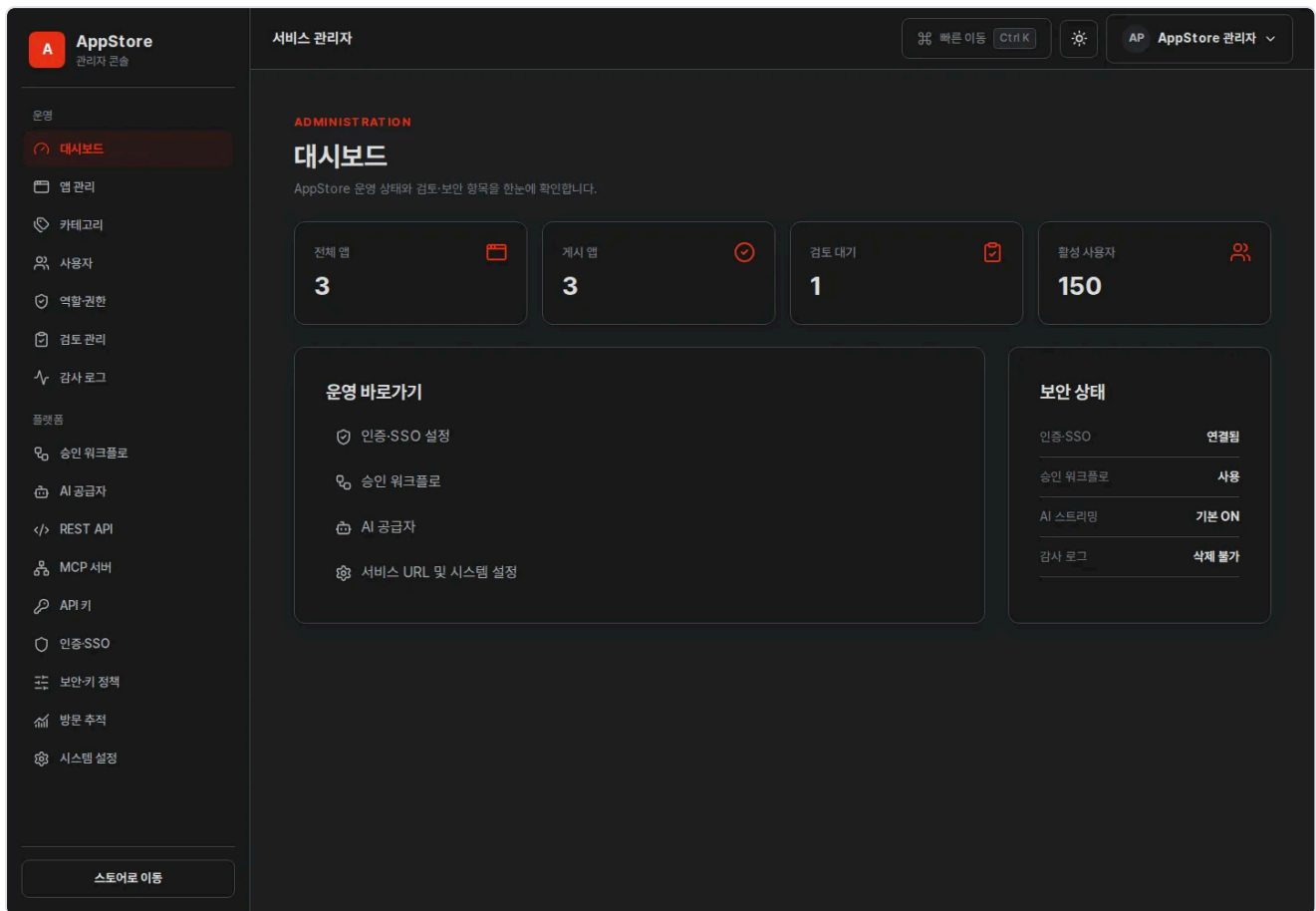
OIDC를 아직 설정하지 않은 상태에서는 BOOTSTRAP_ADMIN / BOOTSTRAP_ADMIN_PASSWORD 로 만들어진 로컬 Super Admin으로 로그인합니다. 로그인 화면의 관리자 계정으로 로그인을 펼치거나 /admin/bootstrap 주소로 바로 들어갑니다.



최초 관리자 로그인 — SSO 설정 전에 쓰는 복구용 관리자 로그인 화면

화면 설명 그대로 SSO를 사용할 수 없을 때를 위한 복구용 관리자 계정입니다. 일상 계정으로 공유하지 마세요. SSO를 붙인 뒤에는 별도의 SSO Super Admin을 지정해 이 계정 의존도를 낮춥니다.

로그인하면 관리자 콘솔 대시보드가 열립니다.



관리자 대시보드 — 앱·사용자 운영 상태와 검토·보안 항목 요약

3. 설정

3.1. 환경 변수 (전수)

애플리케이션 코드가 읽는 환경 변수는 정확히 아래 네 개입니다(`internal/config/config.go`). 이 계약은 `scripts/check-env-contract.sh` 가 CI에서 강제합니다.

이름	기본 값	필 수	설명
POSTGRES_DSN	없음	예	PostgreSQL 연결 문자열. 비어 있으면 기동하지 않습니다. 비밀번호의 예약 문자는 URL 인코딩하세요
BOOTSTRAP_ADMIN	없음	예	최초·복구 Super Admin 식별자. 앞뒤 공백은 잘라 냅니다
BOOTSTRAP_ADMIN_PASSWORD	없음	예	Bootstrap 로그인 비밀번호. 12자 미만이면 기동 자체가 실패합니다
ENCRYPTION_KEY	없음	예	DB에 저장되는 OIDC·AI secret의 암호화 키. 32 raw byte / 64자리 hex / 32 byte의 base64 표현

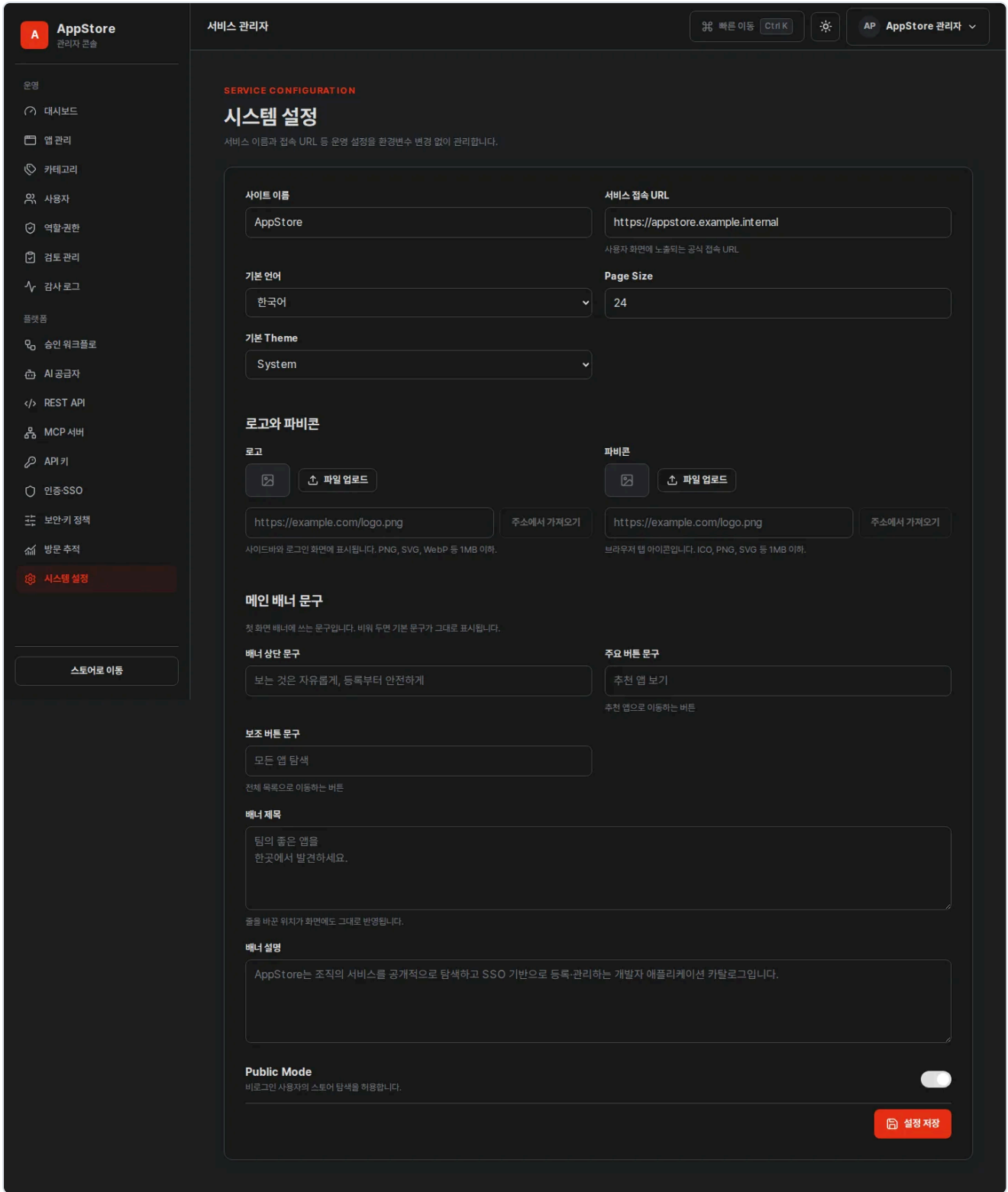
파생 값:

항목	값	바꿀 수 있나
listen 주소	:8080	아니오 — 코드에 고정 (config.ListenAddress). 외부 포트는 -p 로 바꿉니다
요청 헤더 timeout	10초	아니오
요청 읽기 timeout	30초	아니오
idle timeout	2분	아니오
종료 유예	20초	아니오

그 밖의 운영 설정 — 서비스 URL, OIDC, 역할 매핑, 워크플로, 키 정책, AI, REST API, MCP, 브랜딩 — 은 환경 변수가 아니라 관리자 화면과 PostgreSQL에서 관리합니다. 컨테이너를 다시 만들지 않고 바꿀 수 있고, 변경은 감사 로그에 남습니다.

ENCRYPTION_KEY 는 첫 기동에서 지문이 DB에 기록됩니다. 다른 키로 기동하면 ENCRYPTION_KEY does not match the initialized database 로 실패하며 서비스가 뜨지 않습니다. 키를 잃으면 암호화된 OIDC·AI secret은 복원할 수 없습니다.

3.2. 시스템 설정 — 서비스 URL과 배너

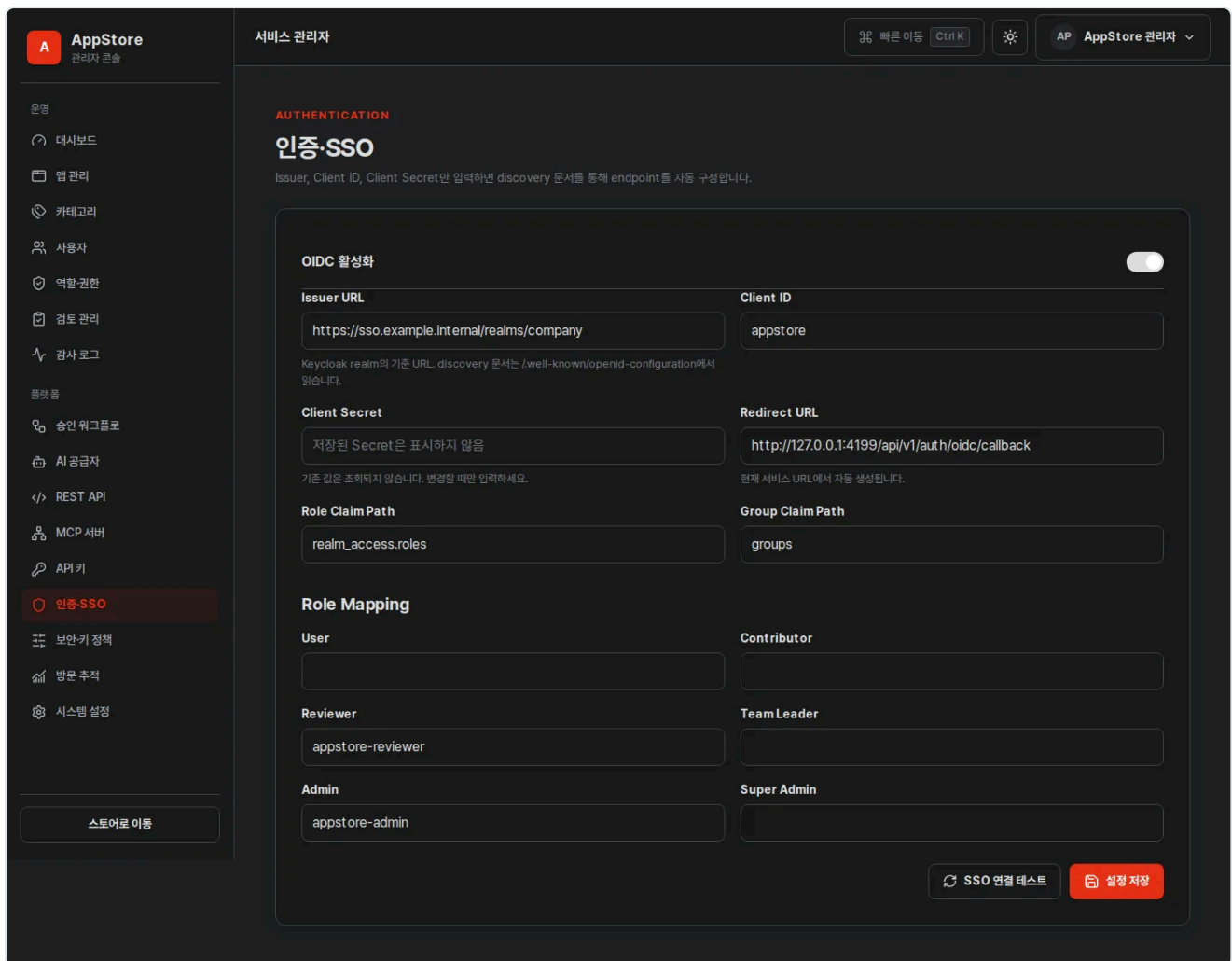


시스템 설정 — 사이트 이름, 서비스 접속 URL, 테마, 공개 모드와 배너 문구

항목	초기값	설명
사이트 이름	Dev App Store	화면 상단과 로그인 화면에 노출
서비스 접속 URL	빈 값	사용자에게 노출되는 공식 HTTPS origin. OIDC Redirect URL이 이 값에서 자동 생성됩니다
기본 Theme	system	light / dark / system
기본 언어	ko	
페이지 크기	24	목록 한 쪽에 보여 줄 앱 수
공개 모드	컴	끄면 비로그인 탐색이 막히고 익명 요청은 <i>익명 AppStore 탐색이 비활성화되어 있습니다.</i> 로 거부됩니다
배너 상단 문구 / 제목 / 설명 / 주요·보조 버튼 문구	빈 값	비워 두면 기본 문구가 그대로 나옵니다. 제목의 줄바꿈은 화면에 그대로 반영됩니다
로고 / 파비콘	없음	파일 업로드 또는 주소에서 가져오기 . 1MB 이하 PNG·JPEG·WebP·GIF·SVG·ICO

로고와 파비콘은 파일 시스템이 아니라 PostgreSQL에 저장됩니다. 이미지를 주소로 가져와도 서버가 한 번 내려받아 저장한 뒤 이 서비스의 origin에서 제공하므로, 업그레이드로 컨테이너를 교체해도 남고 원격 이미지를 막는 콘텐츠 보안 정책에도 걸리지 않습니다. 로고는 로그인 전 화면에도 보이므로 비공개 정보가 담기지 않았는지 확인하세요.

3.3. Keycloak OIDC



Keycloak 쪽 준비:

AppStore 쪽 입력:

칸	초기값	설명
OIDC 활성화	꺼짐	연결 테스트가 성공한 뒤에 켵니다
자동 로그인 (Silent SSO)	꺼짐	Keycloak에 이미 로그인한 사람을 로그인 화면 없이 바로 들여보냅니다. 아래 <i>자동 로그인</i> 참조
Issuer URL	빈 값	realm까지 포함한 기준 URL(예: <code>https://sso.example.internal/realms/company</code>). <code>/.well-known/...</code> 은 붙이지 않습니다
Client ID / Client Secret	빈 값	저장한 Secret은 다시 조회할 수 없습니다. 화면에는 <i>저장된 Secret은 표시하지 않음</i> 만 보입니다
Role Claim Path	<code>realm_access.roles</code>	
Group Claim Path	<code>groups</code>	
Role Mapping	<code>appstore-user</code> → <code>user</code> 등 6개 기본 매핑	아래 4장 참조

연결 테스트는 저장 전 화면에 입력한 Issuer를 그대로 시험합니다. discovery 문서 URL, issuer 일치, authorization·token·userinfo·logout·JWKS endpoint와 PKCE 지원 여부를 확인합니다. 실패 메시지는 원인을 그대로 돌려줍니다.

실패 메시지	원인과 조치
<code>...에 연결하지 못했습니다</code>	폐쇄망에서 컨테이너가 Keycloak에 닿지 못하거나 TLS를 신뢰하지 못합니다
<code>... 요청이 HTTP 404를 반환했습니다</code>	realm 경로 오타. Issuer에 realm이 포함되어야 합니다
discovery 문서에 필수 항목이 없습니다: <code>...</code>	응답이 discovery 문서가 아닙니다. reverse proxy가 로그인·오류 페이지를 대신 주고 있는지 확인
discovery 문서의 issuer(...)가 입력한 Issuer URL(...)과 다릅니다	Keycloak이 내부 hostname으로 issuer를 발행합니다. <code>KC_HOSTNAME</code> 을 사용자에게 노출되는 주소로 맞춥니다

Client Secret과 refresh token은 브라우저에 저장되지 않습니다. 세션 쿠키는 서버가 Secure·HttpOnly로 관리합니다.

자동 로그인 (Silent SSO)

사내 앱을 여럿 오가는 사람이 앱마다 같은 로그인 화면을 지나지 않도록, OIDC의 `prompt=none` 으로 **조용히** 로그인을 시도하는 기능입니다. 기본값은 꺼짐이며, 켜기 전까지는 아무것도 달라지지 않습니다.

켜면 이렇게 동작합니다.

1. 로그인하지 않은 브라우저가 화면을 열면 SPA가 `/api/v1/auth/oidc/login?prompt=none&returnTo=<원래 경로>`로 **최상위 이동**합니다. 숨은 iframe을 쓰지 않으므로 서드파티 쿠키가 막힌 브라우저에서도 동작하고 Keycloak이 프레임 허용하는지 신경 쓰지 않아도 됩니다.
2. Keycloak에 세션이 있으면 인가 코드가 곧바로 돌아와 평소 로그인과 같은 절차로 세션이 만들어지고, 사람은 **처음 열었던 경로로 돌아갑니다**(`returnTo` 는 `/` 로 시작하고 `//` 로 시작하지 않는 값만 받습니다).
3. 세션이 없으면 Keycloak이 화면을 그리지 않고 `error=login_required` 로 돌아옵니다. 이것은 실패가 아니라 평범한 대답이라, 콜백은 `/login?sso=none` 으로 보내 로그인 화면을 보여 줍니다.

같은 시도가 반복되면 브라우저가 Keycloak과 AppStore 사이를 끝없이 오가므로, 되풀이를 막는 장치가 세 겹 있습니다 — 탭 세션당 한 번만 시도(`sessionStorage`), 스스로 로그아웃한 뒤에는 시도하지 않음(다시 로그인하면 풀림), 거절당한 주소의 `sso=none` 표시. 브라우저 저장소를 읽지 못하는 사생활 보호 모드에서는 "이미 시도했다"로 간주해 시도하지 않습니다. 로그인·오류 화면과 `/api` · `/mcp` · `/health` 경로에서는 시도하지 않습니다.

서버는 이 설정이 꺼져 있으면 주소에 `?prompt=none` 이 붙어 와도 조용히 평범한 로그인으로 바꿉니다. 리다이렉트가 생기는 자리는 관리자 설정에만 묶입니다.

공개 모드에서 Keycloak 계정이 없는 익명 방문자는 탭당 한 번 로그인 화면을 거친 뒤에야 카탈로그로 돌아옵니다. 외부 방문자가 많은 공개 카탈로그라면 이 설정을 끄는 편이 맞습니다.

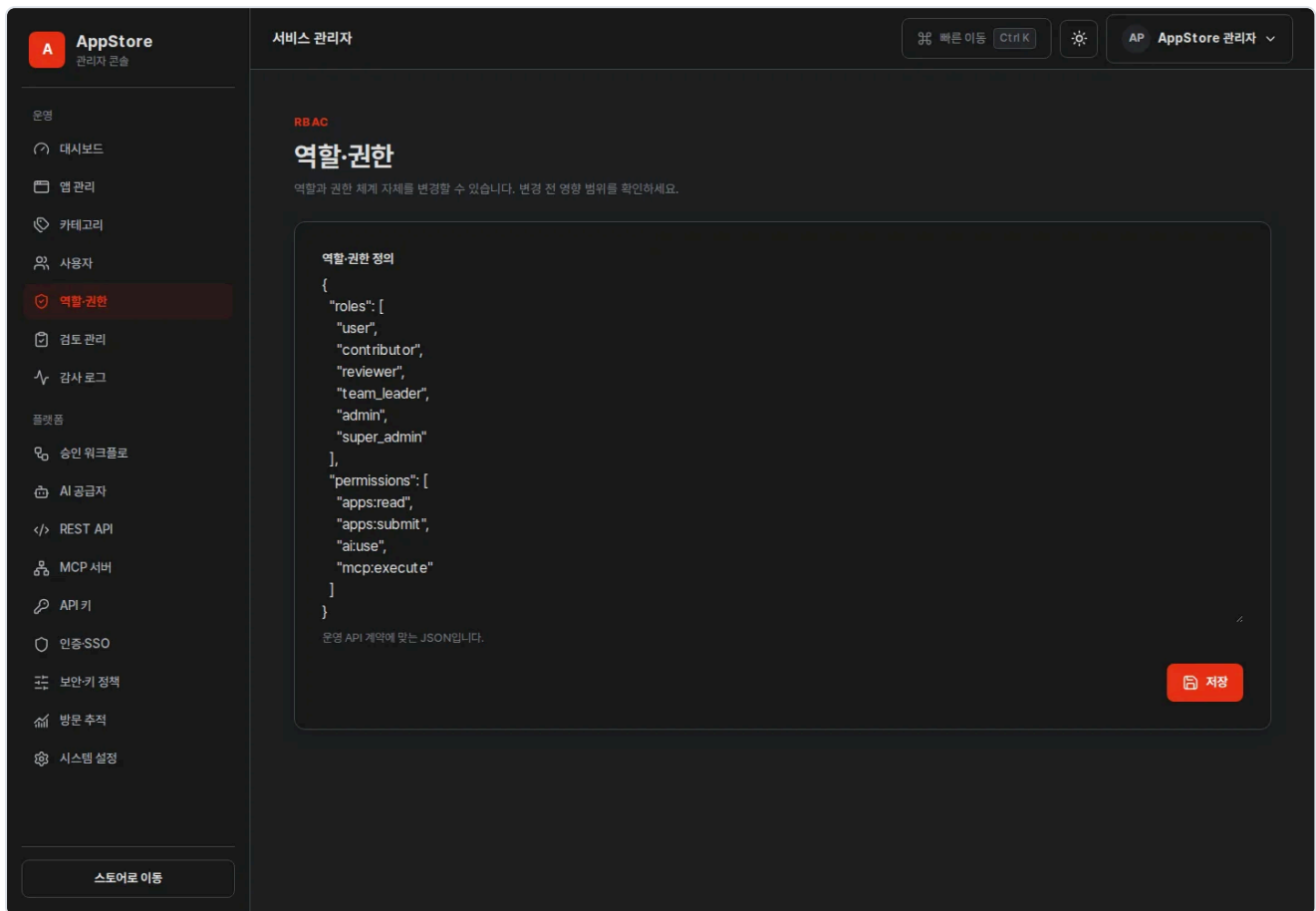
4. 계정과 권한

4.1. 역할

기본 역할 6개와 각 역할이 받는 권한입니다(`migrations/000001_init.sql` 의 시드 값).

역할	key	기본 권한
User	<code>user</code>	<code>apps:read</code> , <code>favorites:read</code> , <code>favorites:write</code> , <code>keys:manage</code> , <code>mcp:read</code>
Contributor	<code>contributor</code>	User + <code>apps:write</code> , <code>apps:update</code> , <code>apps:delete</code> , <code>apps:submit</code> , <code>ai:use</code> , <code>mcp:execute</code>
Reviewer	<code>reviewer</code>	<code>apps:read</code> , <code>reviews:read</code> , <code>reviews:decide</code> , <code>mcp:read</code>
Team Leader	<code>team_leader</code>	Reviewer와 동일. 다만 자기 팀의 검토 건만 보고 처리합니다
Admin	<code>admin</code>	전체 권한
Super Admin	<code>super_admin</code>	전체 권한. Bootstrap·복구 관리자

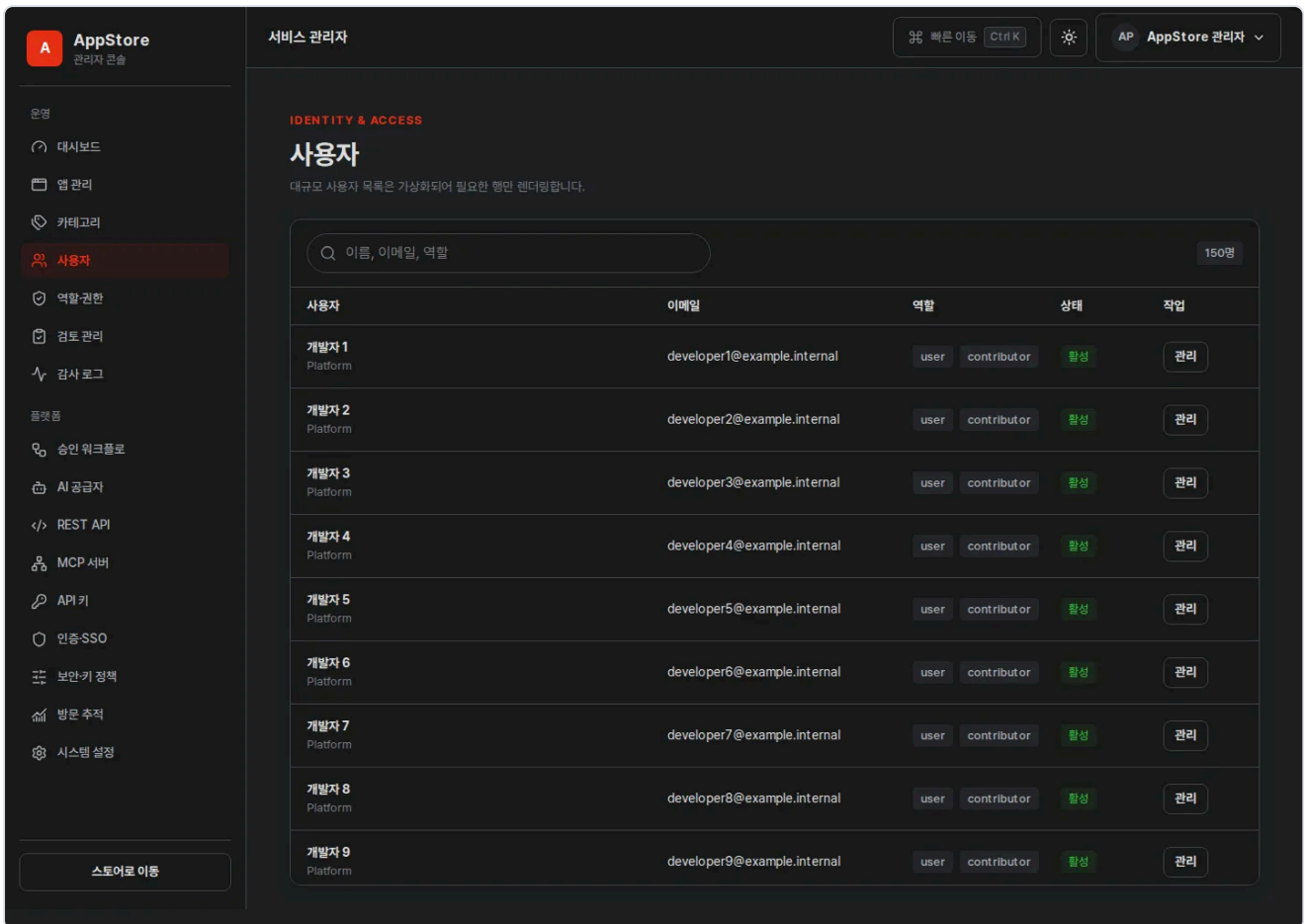
권한 key 전체: apps:read · apps:write · apps:update · apps:delete · apps:submit · apps:manage · favorites:read · favorites:write · reviews:read · reviews:decide · keys:manage · ai:use · mcp:read · mcp:execute · users:manage · roles:manage · settings:read · settings:write · audit:read.



역할·권한 — 역할과 권한 체계 자체를 편집하는 화면

역할과 권한 정의는 화면에서 JSON으로 편집할 수 있습니다. 기본 시스템 역할은 key를 바꿀 수 없고 삭제할 수 없습니다(기본 시스템 역할의 key는 변경할 수 없습니다. / 기본 시스템 역할은 삭제할 수 없습니다.). 바꾸기 전에 영향 범위를 확인하세요.

4.2. 사용자



사용자 — 프로필, 활성 상태와 AppStore 역할을 변경하는 화면

이름·이메일·역할로 검색하고 역할을 바꿉니다. 대량 목록은 가상화되어 필요한 행만 렌더링합니다. 안전장치가 있습니다 — **현재 로그인한 계정은 비활성화할 수 없습니다.**, **현재 로그인한 계정은 삭제할 수 없습니다.**, **Bootstrap 관리자는 삭제할 수 없습니다.**

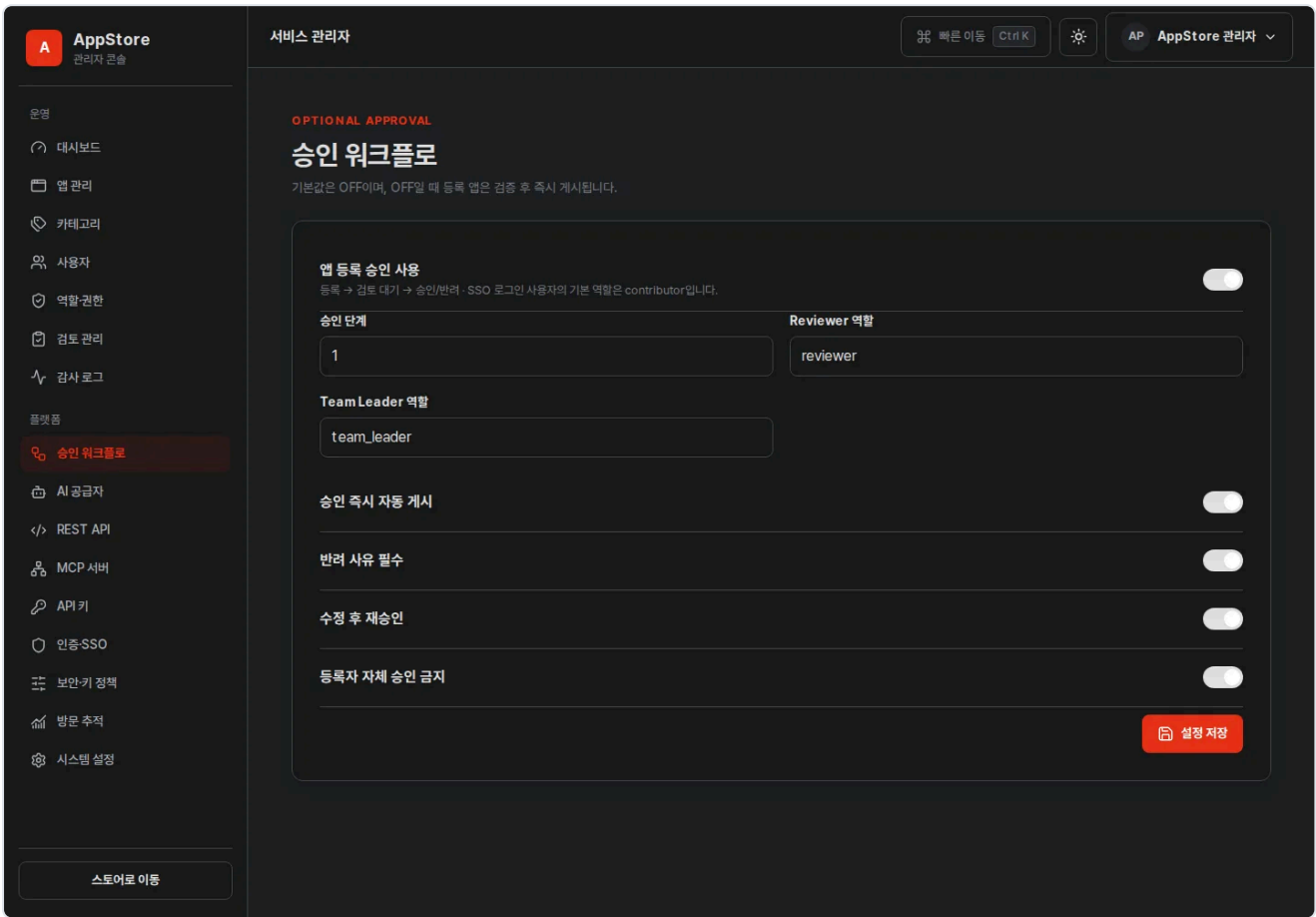
4.3. SSO 역할 매핑

Keycloak 역할 이름은 코드에 고정돼 있지 않습니다. 인증 · SSO 화면의 Role Mapping에서 외부 값을 AppStore 역할에 붙입니다. 초기값은 다음과 같습니다.

AppStore 역할	기본 외부 역할
User	appstore-user
Contributor	appstore-contributor
Reviewer	appstore-reviewer
Team Leader	appstore-manager
Admin	appstore-admin
Super Admin	appstore-super-admin

매핑을 저장한 뒤 각 역할의 시험 계정으로 /submit , /review , /admin 을 실제로 열어 확인하세요. 401 (로그인 필요) 과 403 (권한 거부) 을 구분해 기록합니다.

4.4. 승인 워크플로



승인 워크플로 — 검토 단계, 자동 게시, 반려 사유 필수 등 승인 정책

설정	초기값	뜻
앱 등록 승인 사용	꺼짐	꺼져 있으면 등록이 곧 게시입니다
승인 단계	1 (1~10)	단계마다 검토자가 한 번씩 승인해야 합니다
검토자 역할	["reviewer"]	
팀장 역할	["team_leader"]	팀장 검토에는 사용자 팀 정보가 필요합니다
승인 즉시 자동 게시	끔	마지막 승인 직후 게시할지
반려 사유 필수	끔	등록자가 무엇을 고쳐야 하는지 알 수 있게 합니다
수정 후 재승인	끔	게시된 앱을 고치면 다시 검토를 받습니다
등록자 자체 승인 금지	끔	등록자와 승인자를 분리합니다

기본 역할이 함께 움직입니다 — 승인을 **켜면** SSO 로그인 사용자의 기본 역할이 contributor 가 되어 누구나 제출할 수 있되 게시 전에 검토를 거칩니다. **꺼져 있으면** 제출이 곧 게시이므로 기본 역할은 조회 위주의 user 로 유지되고 등록 권한은 역할 매핑으로만 부여됩니다. 기존 사용자에게는 다음 SSO 로그인부터 반영됩니다.

승인이 켜져 있으면 반려된 앱은 소유자가 내용을 고쳐 저장하는 것만으로 다시 1단계 검토 대기로 돌아갑니다. 별도 재 제출 조작이 없습니다.

검토자가 보는 화면(/review/{id})에는 결정에 필요한 것이 함께 실립니다 — 앱 카드와 상세 설명·태그·스크린샷, 첨부된 가이드 문서(내려받아 확인), 보안 심의 상태와 심의 번호·SecCheck 링크, 등록자·담당팀·검토 단계·버전·서비스 URL, 그리고 같은 앱의 이전 검토와 그때 남긴 의견(최대 20건)입니다. 앱이나 보안 심의 정보를 불러오지 못해도 나머지는 그대로 보입니다.

검토 의견은 승인과 반려 모두에 남길 수 있습니다. 반려는 사유가 필수이고(위 표의 **반려 사유 필수** 설정과 별개로 화면에서 먼저 막습니다), 승인은 선택입니다. 남긴 의견은 등록자의 내 앱·앱 수정 화면과 다음 단계 검토자의 이전 검토 목록에 그대로 보이며, 승인·반려 모두 감사 로그(app.approve , app.reject)에 남습니다.

AppStore
관리자 콘솔

운영

대시보드

앱 관리

카테고리

사용자

역할 권한

검토 관리

감사 로그

플랫폼

승인 워크플로

시 공급자

REST API

MCP 서버

API 키

인증 SSO

보안 키 정책

방문 추적

시스템 설정

스토어로 이동

서비스 관리자

빠른 이동

AppStore 관리자

TEAM REVIEW

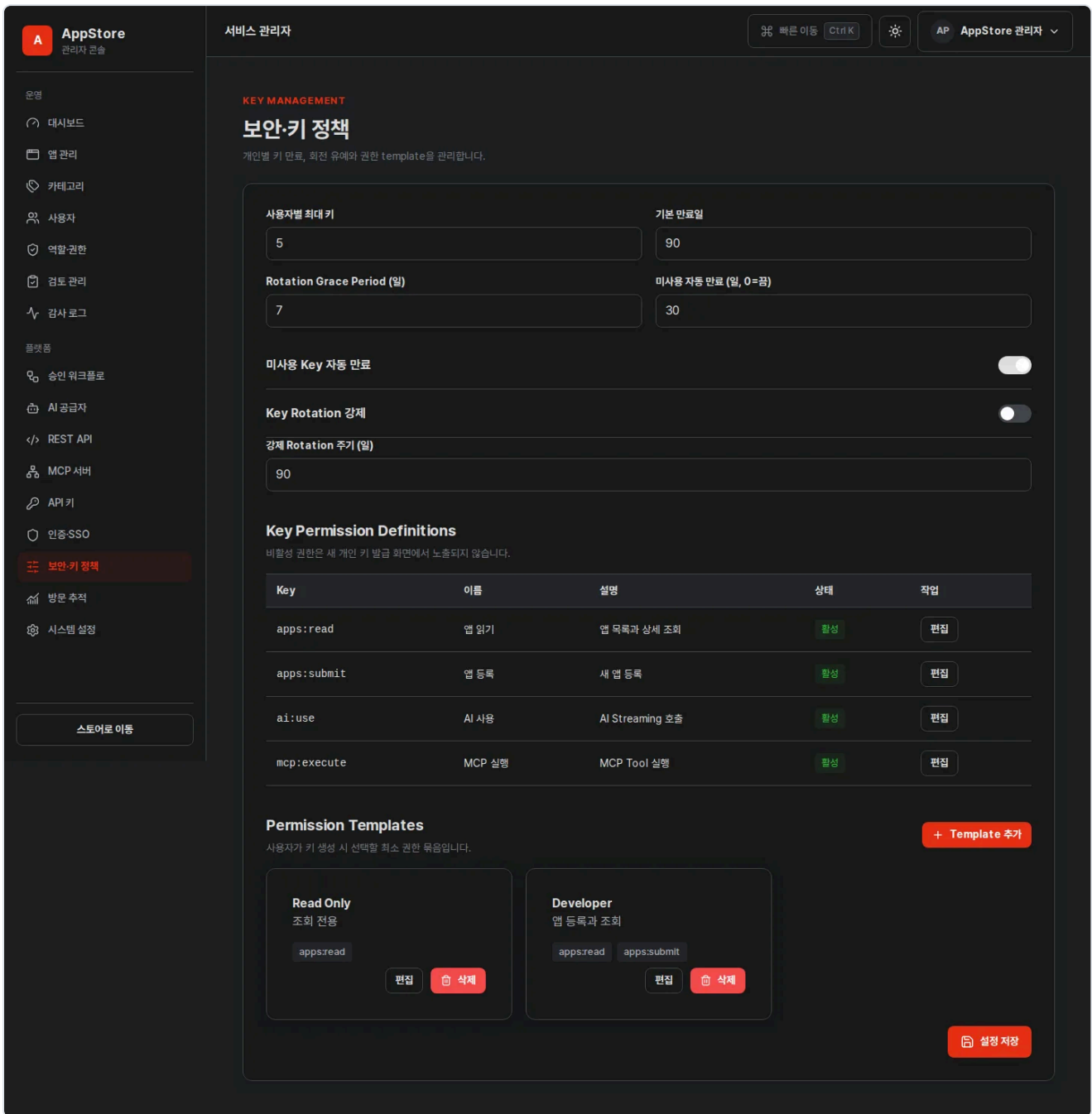
검토 대기

Workflow가 활성화된 경우에만 등록 건이 이 목록에 나타납니다.

앱	등록자	팀	상태	요청 시각	검토
FlowStudio	이자동	Developer Experience	pending	2026. 9. 1. 오후 4:00	열기

검토 관리 — 서비스 전체의 앱 검토 현황

4.5. 개인 키 정책



보안·키 정책 — 사용자별 최대 키, 만료, 회전 유예와 권한 템플릿

화면의 칸	초기값	허용 범위
사용자별 최대 키	5	1~100
기본 만료일	90	1~3650
Rotation Grace Period (일)	7	0~365
미사용 Key 자동 만료 / 미사용 자동 만료 (일, 0=끔)	꺼짐 / 90	1~3650
Key Rotation 강제 / 강제 Rotation 주기 (일)	꺼짐 / 90	1~3650

같은 화면에서 Key 권한 정의(사용자가 키에 붙일 수 있는 권한)와 권한 템플릿을 관리합니다. 비활성화한 권한은 새 키 발급 화면에 나타나지 않고, 템플릿에는 활성 권한만 넣을 수 있습니다(**활성화된 Key 권한만 Template에 포함할 수 있습니다**). 초기 템플릿은 Read Only · Developer · AI Client · MCP Client · Full Access입니다.

AppStore
관리자 콘솔

운영

대시보드

앱 관리

카테고리

사용자

역할 권한

검토 관리

감사 로그

플랫폼

승인 워크플로

AI 공급자

REST API

MCP 서버

API 키

인증 SSO

보안 키 정책

방문 추적

시스템 설정

스토어로 이동

서비스 관리자

앱 빠른 이동

Ctrl K

🌞

AP AppStore 관리자

KEY INVENTORY

API 키

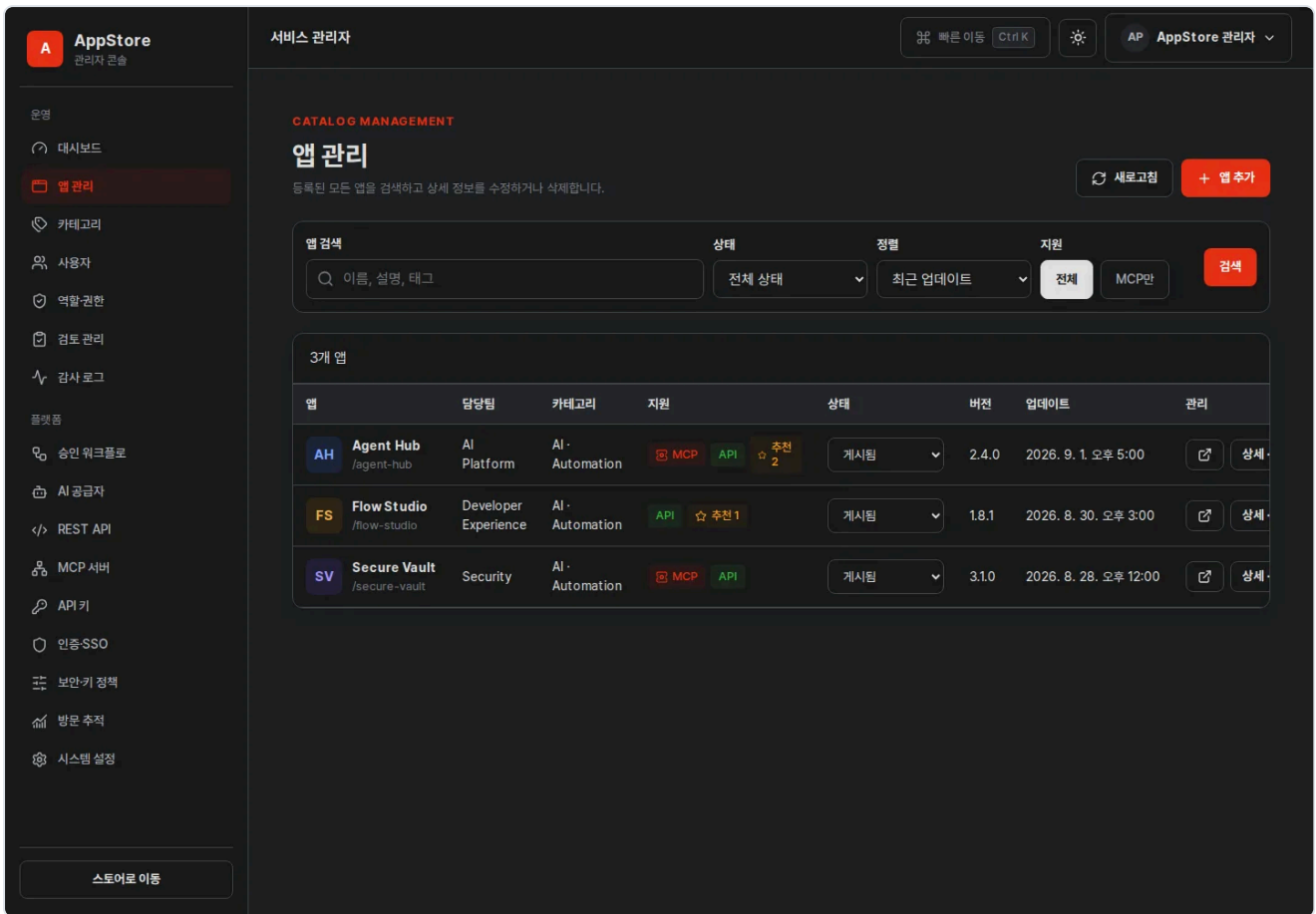
원문 없이 prefix, 소유자, 권한과 사용 상태만 조회합니다.

소유자	이름	Prefix	권한	최근 사용	상태
개발자 1	CI	aps_demo****	1	—	활성

API 키 — 서비스 전체 키의 prefix, 소유자, 권한과 사용 상태

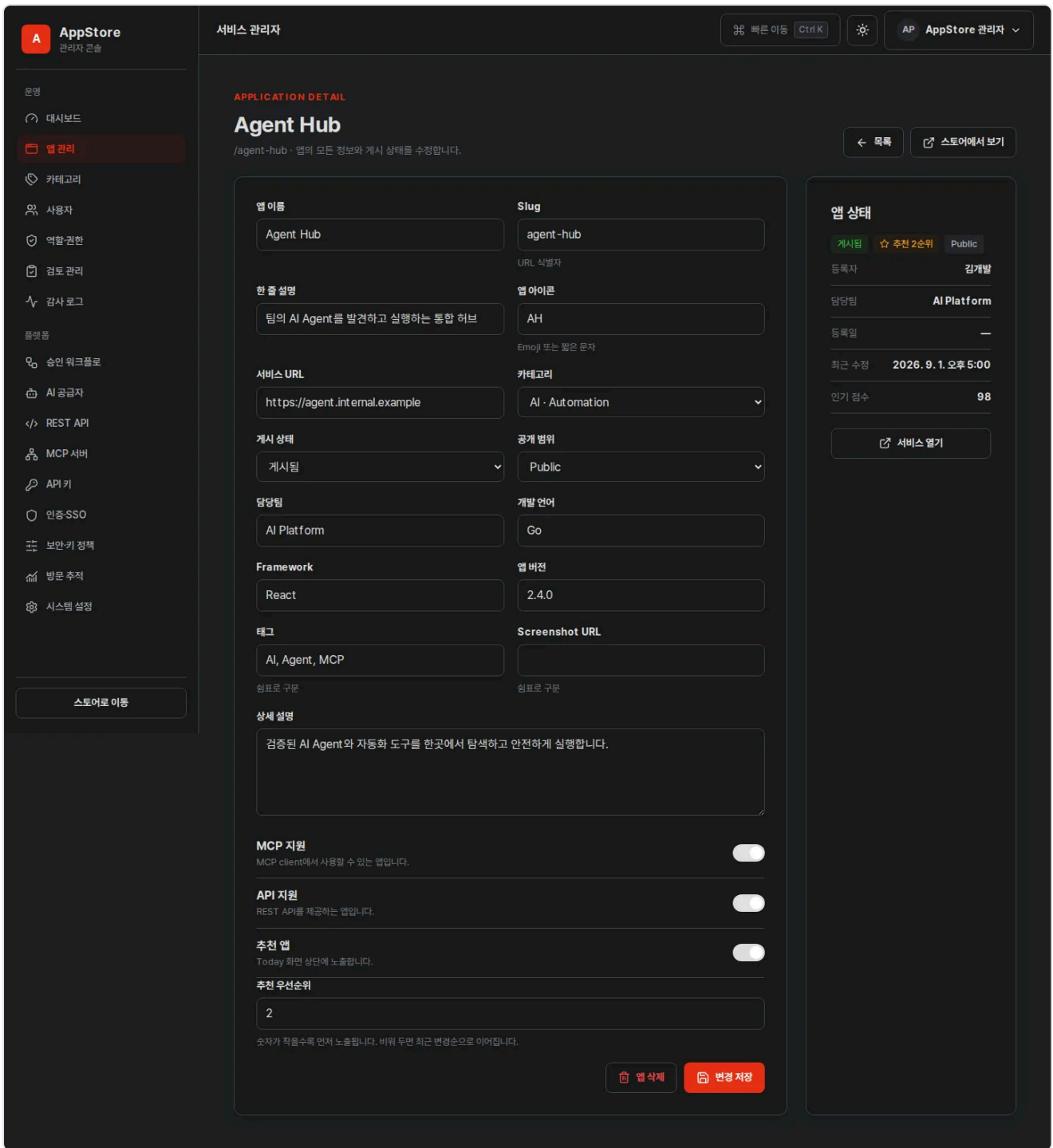
관리자 화면에서도 키 원문은 보이지 않습니다. **원문 없이 prefix, 소유자, 권한과 사용 상태만 조회합니다**. DB에는 prefix와 keyed hash만 저장됩니다.

4.6. 카탈로그 관리



앱 관리 — 등록된 모든 앱을 검색하고 상태를 바꾸는 화면

이름·설명·태그 검색, 게시 상태, 지원 기능, 정렬로 좁힙니다. 조건은 URL에 남습니다. 목록의 상태 select는 게시 상태 만 즉시 바꾸고, 나머지 정보는 상세 · 수정에서 편집합니다.



앱 상세 — 앱의 모든 정보와 게시 상태를 수정하는 관리자 화면

- **추천 우선순위** — 숫자가 작을수록 투데이의 추천 진열에서 먼저 나옵니다. 비워 두면 최근 변경순으로 이어집니다.
- 게시를 멈추기만 하려면 상태를 **보관됨**으로 바꿉니다. 카탈로그에서 내려가지만 기록은 남습니다.
- **가이드 문서** — 앱마다 사용 설명서를 첨부합니다. PDF, Markdown, 텍스트, CSV, Word, PowerPoint, Excel, 한글 문서만 받고 파일당 20MB, 앱당 10개, 같은 파일 이름은 한 번까지입니다. 첨부과 삭제는 **변경 저장**을 눌러야 적용되고, 소유자와 `apps:manage` 권한을 가진 관리자만 바꿀 수 있습니다. 게시된 공개 앱의 문서는 **로그인 없이 누구나** 내려받으므로 사내 공개 범위의 문서만 올리도록 안내하세요. 미게시·비공개 앱의 문서는 소유자·관리자·검토자에게만 보입니다.

- **내장 가이드** — 이미지는 사내 서비스별 `USER_GUIDE.pdf` · `ADMIN_GUIDE.pdf` 가 실려 있고, 기동할 때마다 디렉터리 이름과 slug를 각각 소문자로 바꿔 같은 앱에만 *사용자 가이드*·*관리자 가이드*로 첨부됩니다. 같은 파일 이름의 문서가 이미 있으면 내용이 바뀐 것만 제자리에서 교체하고(문서 ID와 내려받기 주소는 그대로), 같으면 손대지 않으며, 다른 이름으로 올린 문서는 건드리지 않습니다. 앱이 이미 10개를 다 채웠거나 아직 등록되지 않은 slug는 건너뛰고 로그에 남깁니다. 자동으로 붙은 문서는 올린 사람이 비어 있으며, 지워도 다음 기동에서 다시 붙습니다. 내장 가이드를 갱신하는 방법은 README의 `make guides` 를 참고하세요.
- **앱 삭제**는 영구 삭제입니다. *앱과 함께 즐겨찾기, 검토 이력, 버전 기록이 모두 삭제되며 되돌릴 수 없습니다.* 첨부된 가이드 문서도 같이 사라집니다.

A

AppStore

관리자 콘솔

운영

대시보드

앱 관리

카테고리

사용자

역할 권한

검토 관리

감사 로그

플랫폼

승인 워크플로

AI 공급자

REST API

MCP 서버

API 키

인증 SSO

보안 키 정책

방문 추적

시스템 설정

스토어로 이동

서비스 관리자

앱 빠른 이동

Ctrl K

🌞

AP AppStore 관리자

NEW APPLICATION

앱 추가

카탈로그에 앱을 직접 등록합니다. 검토 절차 없이 선택한 게시 상태로 저장됩니다.

← 목록

앱 이름

Slug

한 줄 설명

앱 아이콘

서비스 URL

카테고리

게시 상태

공개 범위

담당팀

개발 언어

Framework

앱 버전

태그

Screenshot URL

상세 설명

MCP 지원

API 지원

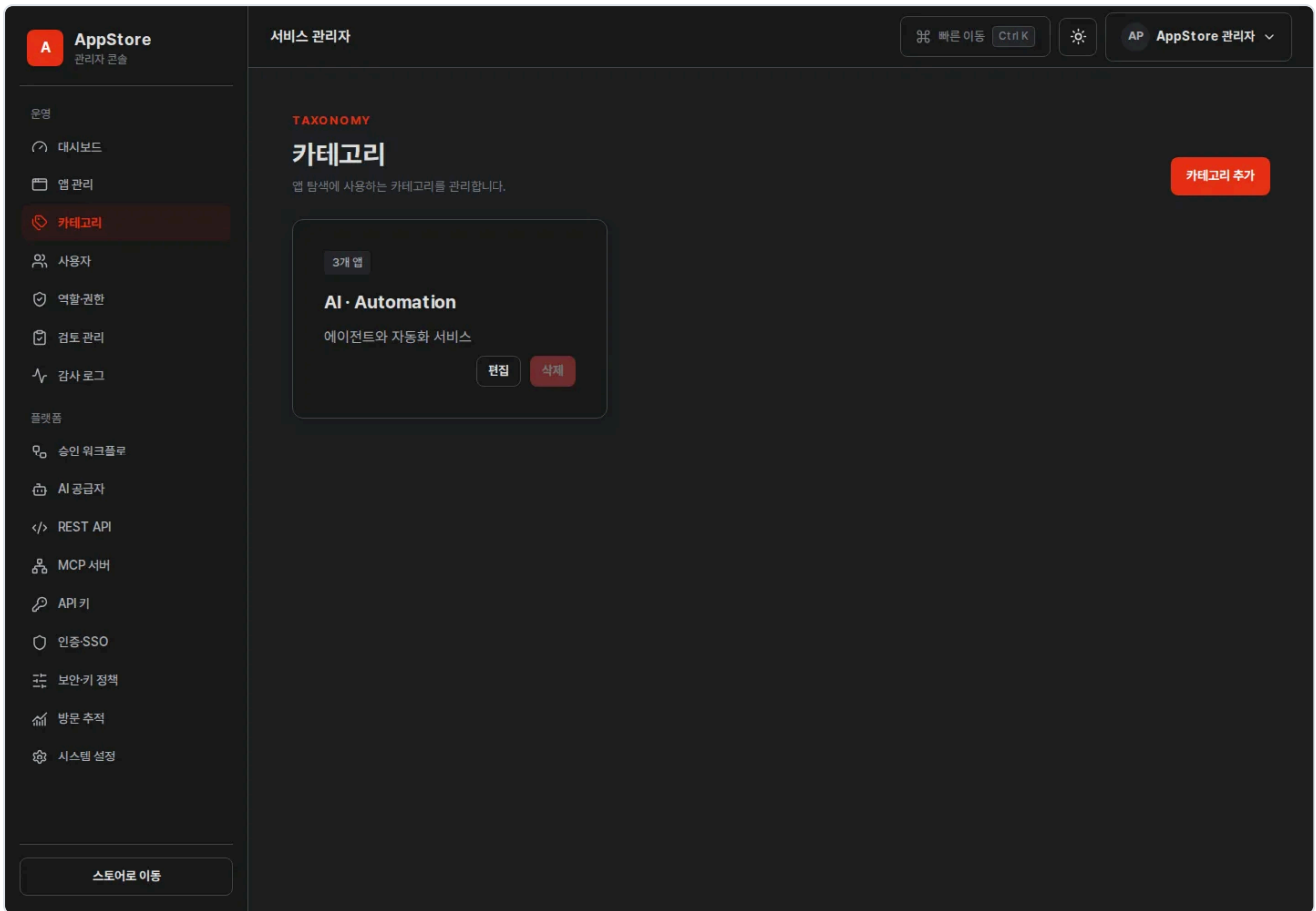
추천 앱

추천 우선순위

앱 등록

관리자 앱 등록 — 검토 절차 없이 카탈로그에 앱을 직접 등록

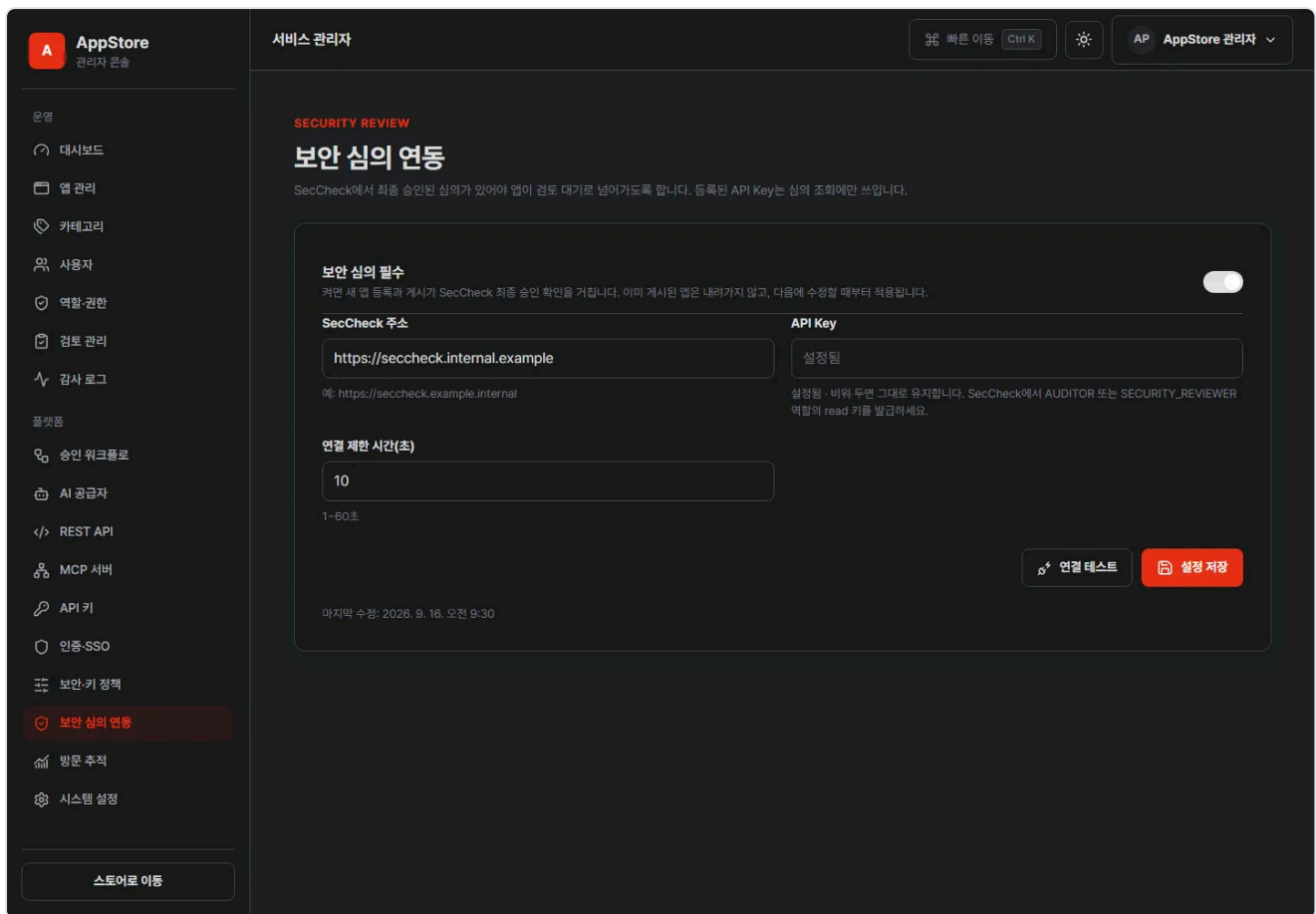
관리자는 검토 절차를 거치지 않고 선택한 게시 상태로 바로 등록할 수 있습니다.



카테고리 — 앱 탐색에 쓰는 카테고리 관리

카테고리는 식별자(slug), 이름, 아이콘, 설명, 정렬 순서를 받습니다. 식별자와 이름은 비워 둘 수 없고, 아이콘을 비우면 📦가 기본값으로 들어갑니다. **사용 중인 카테고리는 삭제할 수 없습니다.**

4.7. 보안 심의 연동 (SecCheck)



보안 심의 연동 — SecCheck 주소와 API Key, 보안 심의 필수 설정

Admin → 보안 심의 연동 (/admin/security-check)에서 SecCheck를 연결합니다. 기본값은 꺼짐이고, 켜기 전까지 등록 절차는 예전과 똑같습니다.

항목	설명
보안 심의 필수	켜면 앱 등록·제출·게시가 SecCheck 최종 승인 확인을 거칩니다
SecCheck 주소	예: <code>https://seccheck.example.internal</code> . 인증정보·쿼리가 붙은 주소는 저장되지 않습니다
API Key	SecCheck에서 발급한 <code>read</code> 개인 API Key. <code>AUDITOR</code> 또는 <code>SECURITY_REVIEWER</code> 역할이 있어야 심의를 조회할 수 있습니다
연결 제한 시간	1~60초. SecCheck 응답을 기다리는 시간입니다

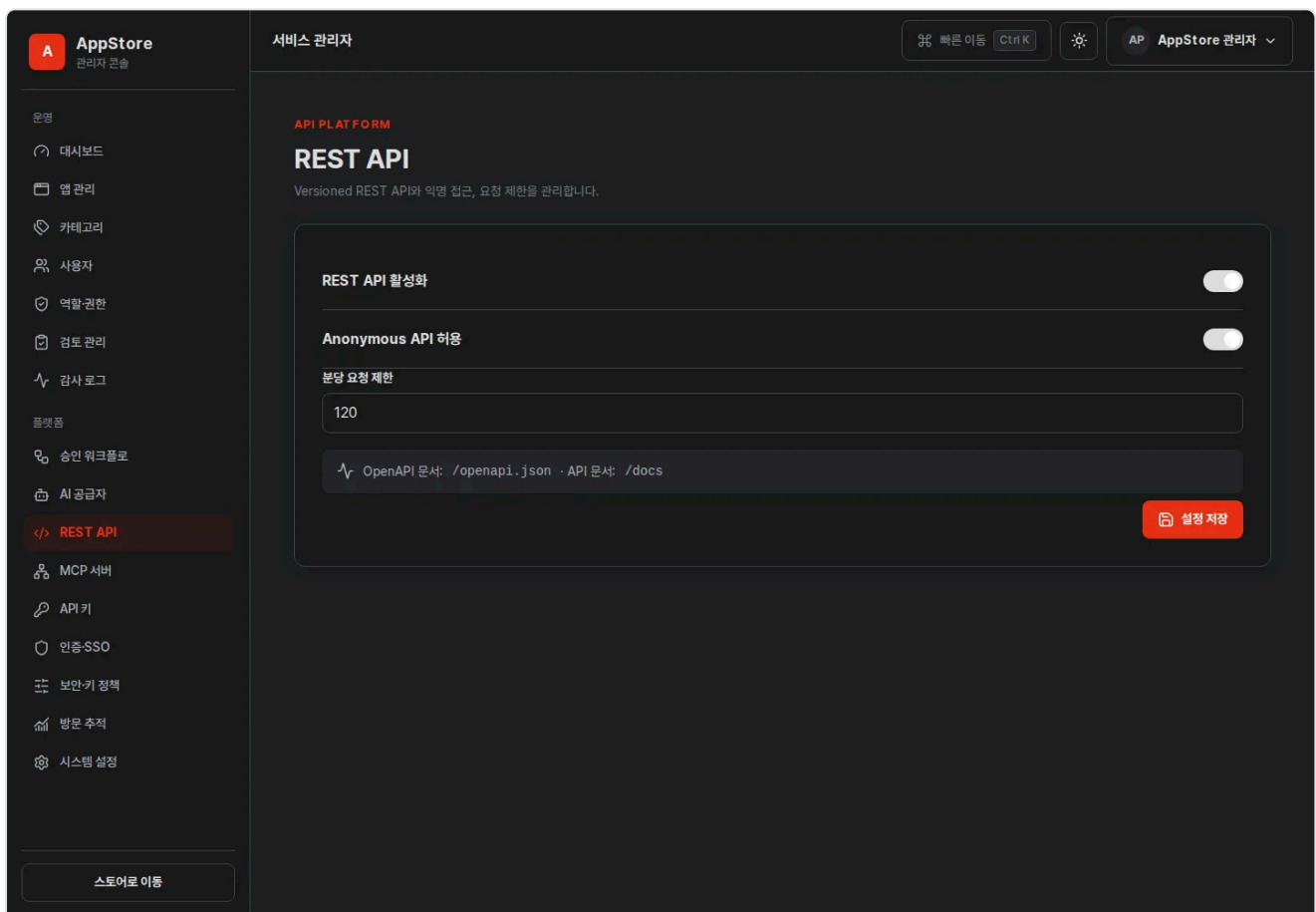
연결 테스트는 저장된 주소와 키로 SecCheck의 `/api/v1/me` 를 조회해 계정과 역할을 확인합니다. API Key는 저장 뒤 화면으로 돌려주지 않고 설정됨으로만 표시되며, `ENCRYPTION_KEY` 로 암호화되어 저장됩니다. 주소를 바꿀 때는 그 서비스의 키를 함께 입력해야 합니다 — 이전 서비스의 키가 새 주소로 전송되지 않게 하기 위해서입니다.

동작은 이렇습니다.

- 등록자가 앱을 등록하면 앱은 초안으로 남고 검토 대기로 가지 않습니다. 등록자는 앱마다 발급되는 연동 정보를 SecCheck 심의 설명에 붙여 넣고, 심의 ID로 승인 결과를 확인한 뒤 제출합니다.
- AppStore는 심의를 서버에서 직접 조회합니다. 등록자가 보내는 값은 심의 ID뿐이고, 승인 여부·서비스명·미처리 항목 수는 모두 SecCheck의 응답에서 읽습니다.
- 승인으로 인정하는 조건은 **status=APPROVED, final_result=APPROVED, 미검토·미확인 보완·판정 후 변경이 모두 0건**입니다. 조건부 승인(CONDITIONAL)과 종료(CLOSED)는 통과하지 못합니다.
- 앱을 수정하면 연동 정보가 새로 발급되어 이전 승인은 무효가 되고, 앱은 다시 초안으로 돌아갑니다. 관리자도 승인 없이는 게시나 검토 대기로 상태를 바꿀 수 없습니다.
- 이미 게시된 앱은 이 설정을 켜도 내려가지 않습니다. 다음에 수정할 때부터 적용됩니다.
- SecCheck 주소나 키를 바꾸면 이전 연동에서 읽은 승인은 모두 무효가 됩니다.

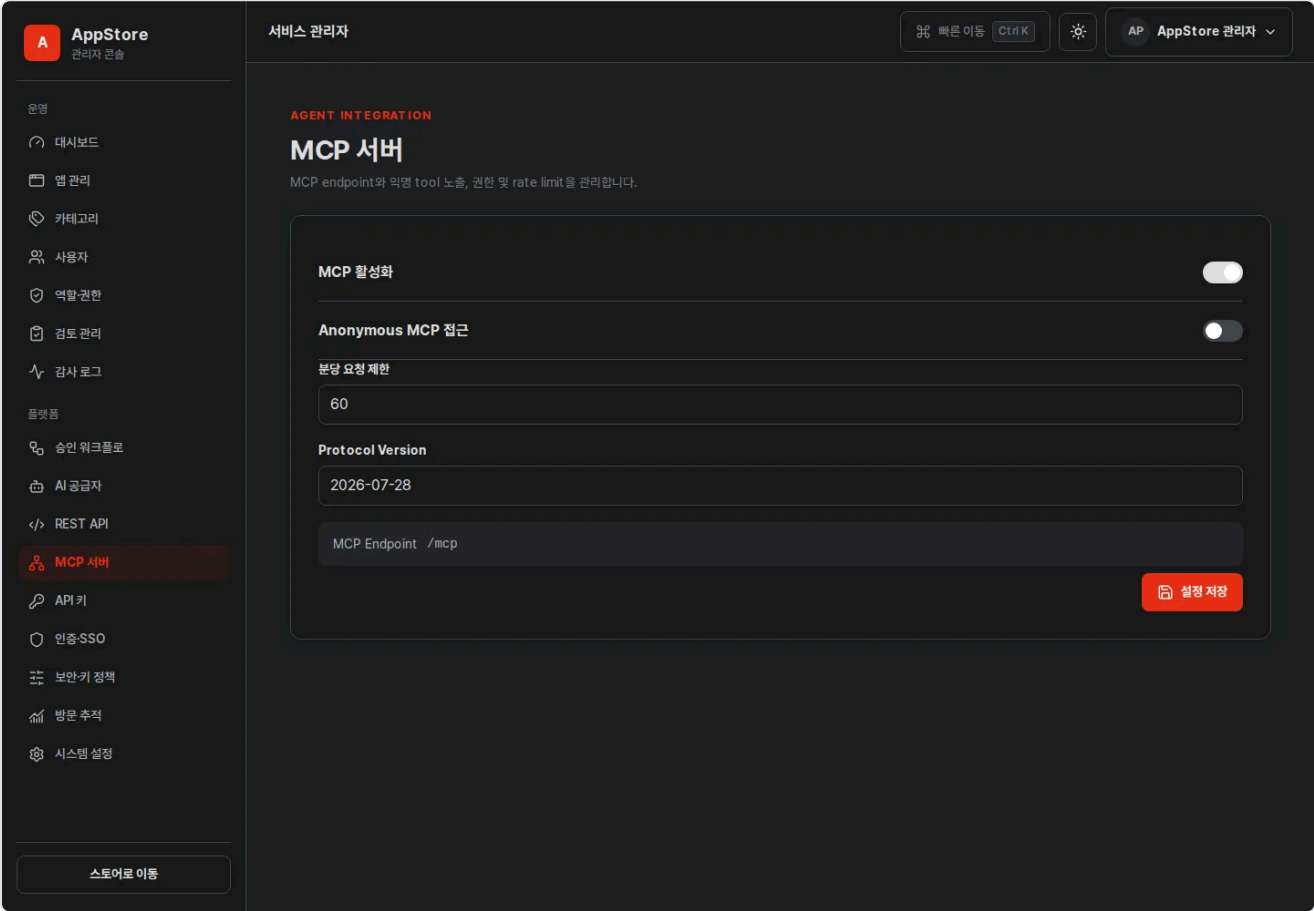
확인과 설정 변경은 모두 감사 로그(`security_check.settings.update` , `security_check.connection.test` , `security_check.verify`)에 남습니다.

4.8. REST API · MCP · AI



REST API — 공개 범위와 분당 요청 제한

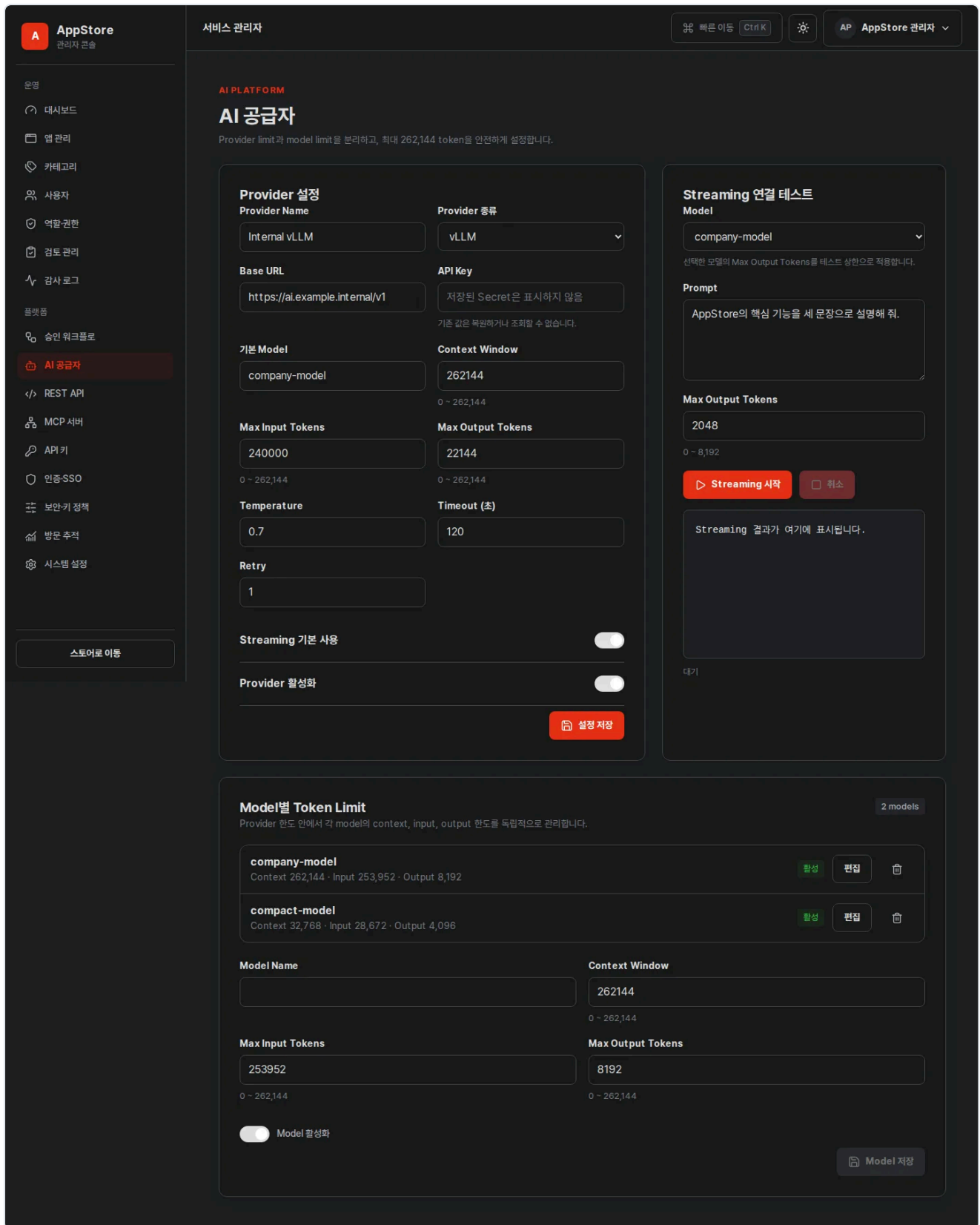
설정	초기값
API 사용	컴
익명 조회 허용	컴
분당 요청 제한	120



MCP 서버 — endpoint 사용 여부, 익명 tool 노출과 rate limit

설정	초기값
MCP 사용	컴
익명 조회 허용	컴
분당 요청 제한	60
Protocol Version	2026-07-28

MCP tool은 호출자의 권한에 따라 다르게 보입니다. 익명·인증 사용자는 조회 tool(`apps_list` , `apps_search` , `app_get` , `categories_list` , `featured_apps` , `trending_apps` , `mcp_apps`)을, 인증 사용자는 자기 앱 tool(`my_apps` , `app_submit` , `app_update`)을, 관리자는 운영 tool(`apps_manage` , `settings_get` , `workflow_get`)을 함께 봅니다.

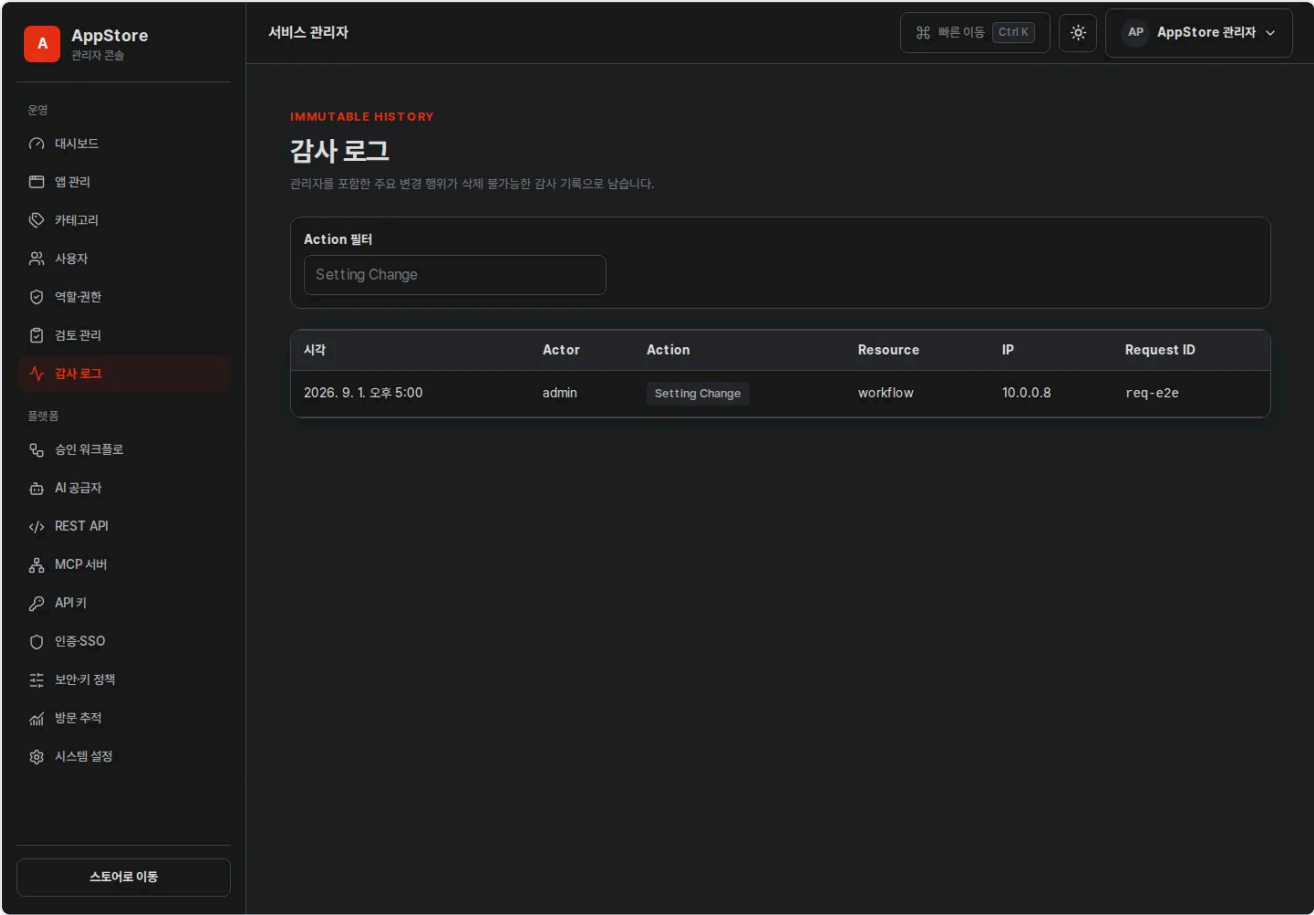


AI 공급자 — provider와 모델별 token 한도, streaming 설정

Provider의 Base URL·API Key·기본 모델과, 모델별 Context Window / Max Input Tokens / Max Output Tokens를 따로 관리합니다. 최대 262,144 token까지 설정할 수 있고, Model의 최대 입력·출력 token 합은 context window를 넘을 수 없습니다. API Key는 ENCRYPTION_KEY 로 암호화되어 저장되고 화면에는 마스크만 보입니다.

Provider 한도와 모델 한도를 분리해 두어야 upstream이 지원하지 않는 값을 강제로 보내지 않습니다.

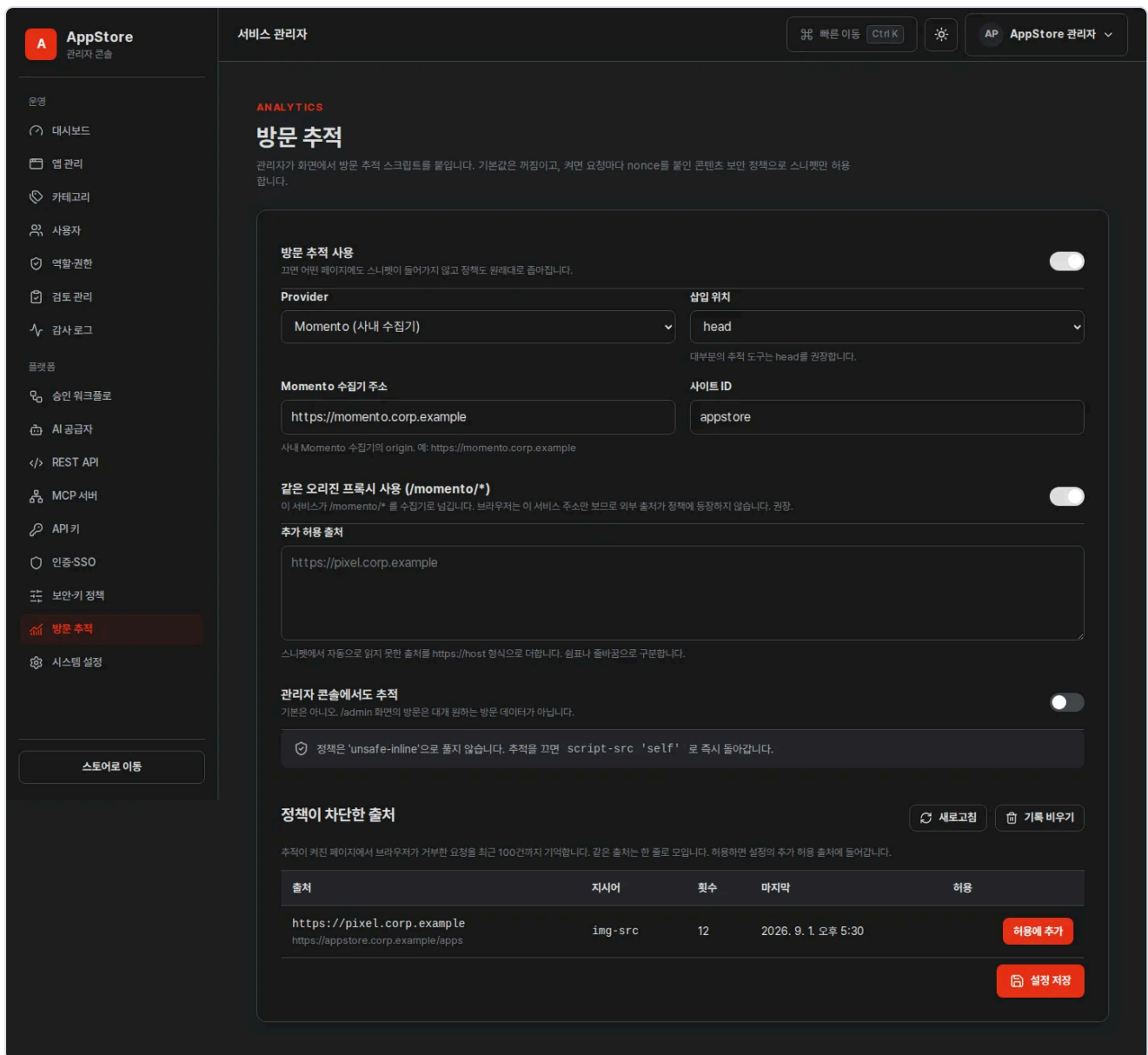
4.9. 감사 로그



감사 로그 — 삭제할 수 없는 관리자 행위 기록

로그인, 앱 변경, 승인·반려, 키 수명주기, 역할·설정 변경이 actor·대상·시각·요청 ID와 함께 남습니다. DB 트리거 (audit_logs_immutable)가 UPDATE와 DELETE를 막으므로 관리자에게도 삭제 기능이 없습니다.

4.10. 방문 추적과 콘텐츠 보안 정책



방문 추적 — provider, Momento 프록시, 허용 출처와 정책이 차단한 출처

관리자 → 방문 추적에서 방문 추적 스크립트를 화면에 붙입니다. 설정은 환경 변수가 아니라 PostgreSQL(system_settings.analytics)에 저장되므로 수집기 주소가 바뀌어도 재배포 없이 바꿀 수 있고, 변경은 감사 로그에 analytics.setting.update 로 남습니다. 기본값은 꺼짐입니다. 새로 설치한 곳은 이 화면을 손대기 전까지 아무것도 달라지지 않습니다.

칸	초기값	설명
방문 추적 사용	꺼짐	고면 어떤 페이지에도 스니펫이 들어가지 않고 정책도 원래대로 좁아집니다
Provider	사용 안 함	Momento · GA4 · GTM · Matomo · 직접 붙여 넣기
Momento 수집기 주소 / 사이트 ID	빈 값	사내 Momento 수집기의 origin과 사이트 id
같은 오리진 프록시 사용	켄	이 서비스가 /momento/* 를 수집기로 넘깁니다. 브라우저는 이 서비스 주소만 보므로 외부 출처가 정책에 등장하지 않습니다
측정 ID / Matomo 주소 · 사이트 ID	빈 값	GA4·GTM·Matomo용. 영문·숫자· _ . : - 64자 이내
추적 스니펫	빈 값	직접 붙여 넣기용 <script> 태그. 8KB를 넘으면 저장되지 않습니다
추가 허용 출처	빈 값	스니펫에서 자동으로 읽지 못한 https://host 를 심표·줄바꿈으로 나열
관리자 콘솔에서도 추적	꺼짐	/admin 화면은 기본으로 추적하지 않습니다
삽입 위치	head	head 또는 body

Momento를 먼저 씁니다. Momento는 사내 자체 호스팅 수집기라 방문 데이터가 밖으로 나가지 않는 유일한 선택지입니다. 같은 오리진 프록시를 켜 두면 스니펫은 /momento/tracker.js 를 읽고 이벤트를 /momento 로 보내며, 서비스가 그 요청을 수집기 주소로 전달합니다. 전달할 때 이 서비스의 세션 쿠키와 Authorization 헤더는 떼어 냅니다 — 수집기는 이벤트만 받고 방문자의 세션은 받지 않습니다. 추적이 꺼져 있거나 provider가 Momento가 아니면 /momento/* 는 404입니다. GA4·GTM은 데이터가 Google로 나가므로 폐쇄망에서는 동작하지 않습니다.

콘텐츠 보안 정책. 모든 화면은 script-src 'self' 로 잠겨 있어 스니펫을 그냥 붙이면 브라우저가 조용히 차단합니다. 추적을 켜면 서비스는 다음을 요청마다 합니다.

1. 무작위 nonce를 만들어 정책의 script-src 에 'nonce-...' 로 넣고, 스니펫의 모든 <script> 태그에 같은 값을 붙입니다. 'unsafe-inline' 은 쓰지 않습니다. 한 번 풀면 그 앱의 모든 인라인 스크립트가 함께 허용되고 추적을 끈 뒤에도 느슨한 채 남기 때문입니다.
2. provider가 정한 출처(예: Matomo 주소)와 붙여 넣은 스니펫에서 읽어 낸 http(s) 출처, 추가 허용 출처를 script-src · connect-src · img-src 에 더합니다.
3. report-uri /api/v1/analytics/csp-report 를 정책에 넣어 브라우저가 거부한 요청을 신고하게 합니다. 이 신고는 추적이 켜진 동안에만 옵니다.

/api/* · /mcp · /health* · /momento/* 같은 비화면 경로는 default-src 'none' 으로 오히려 더 좁고, 관리자 콘솔은 관리자 콘솔에서도 추적을 켜 있을 때만 스니펫을 받습니다. 추적을 끄면 정책은 7.2의 원래 문자열로 즉시 돌아갑니다.

정책이 차단한 출처. 같은 화면 아래에 브라우저가 신고한 차단이 출처·지시어·횟수로 모입니다(메모리에 최근 100건, 같은 출처는 한 줄). 화면이 비어 보이는데 여기에 출처가 있으면 그 출처가 막힌 것입니다. 허용에 추가를 누르면 추가 허용 출처에 들어가고 다음 페이지부터 정책에 실립니다. 스니펫을 고친 뒤에는 기록 비우기로 지우고 다시 띄워 남는 차단이 없는지 확인합니다. Momento 프록시를 쓰면 외부 출처가 없으니 이 표가 비어 있는 것이 정상입니다.

5. 운영

5.1. 상태 점검 엔드포인트

라우트 등록 자리(`internal/httpapi/server.go`)에서 확인한 메서드와 경로입니다.

메서드	경로	응답	용도
GET	<code>/health/live</code> (별칭 <code>/healthz</code>)	<code>{"status":"ok","uptimeSeconds":...}</code>	프로세스 살아 있는지. 컨테이너 HEALTHCHECK가 이 경로를 씁니다
GET	<code>/health/ready</code> (별칭 <code>/readyz</code>)	200 <code>{"status":"ready"}</code> / 503 <code>{"status":"not_ready"}</code>	DB ping(2초 timeout). load balancer는 이 경로를 봅니다
GET	<code>/api/version</code>	version · commit · buildDate	배포된 빌드 확인
GET	<code>/openapi.json</code>	OpenAPI 3.1 문서	
GET	<code>/docs</code>	API 문서 화면	

5.2. 로그

로그는 표준 출력으로 나가는 JSON 한 줄입니다(`slog` JSON handler, level INFO). 파일에 쓰지 않으므로 `docker logs` 또는 조직의 로그 수집기로 봅니다.

```
docker logs --tail 200 appstore
docker compose logs --tail 200 appstore
```

실제로 찍히는 메시지:

msg	함께 나오는 필드	언제
appstore listening	address , version	기동 완료
http request	method , path , status , bytes , duration_ms , request_id	모든 요청
panic recovered	error , request_id , stack	처리 중 패닉. 응답은 500 INTERNAL_ERROR
appstore shutting down	—	SIGINT/SIGTERM 수신, 20초 유예
appstore stopped	error	기동 실패 또는 비정상 종료(종료 코드 1)

사용자가 오류 화면에서 알려 준 **요청 ID**로 `request_id` 를 검색하면 해당 요청 한 건을 찾을 수 있습니다. 요청 ID는 응답의 `X-Request-ID` 헤더로도 나가고, 클라이언트가 보낸 `X-Request-ID` 가 8~128자이면 그 값을 그대로 씁니다.

5.3. 백업

백업 대상은 **PostgreSQL**과 **ENCRYPTION_KEY** 두 가지이고, 서로 다른 저장소에 둡니다. 컨테이너에는 백업할 상태가 없습니다. 로고·파비콘과 앱 가이드 문서도 파일 시스템이 아니라 데이터베이스에 있어 dump 하나로 함께 보존되지만, 그만큼 dump 크기가 첨부한 문서 용량만큼 커집니다.

```
backup_stamp="$(date -u +%Y%m%dT%H%M%SZ)"
pg_dump --format=custom --no-owner --no-privileges \
  --file="appstore-$backup_stamp.dump" "$POSTGRES_DSN"
pg_restore --list "appstore-$backup_stamp.dump" >/dev/null
sha256sum "appstore-$backup_stamp.dump" >"appstore-$backup_stamp.dump.sha256"
```

dump만으로는 완전한 복구가 아닙니다. OIDC Client Secret과 AI API Key는 **ENCRYPTION_KEY** 로 암호화돼 있어 원래 키가 없으면 복호화할 수 없습니다. 키 사본을 승인된 vault에 최소 두 벌 두고, DB 백업 접근자와 키 접근자를 분리하세요.

복원은 격리된 데이터베이스에 먼저 시험합니다.

```
pg_restore --clean --if-exists --no-owner --no-privileges \
  --dbname="$RESTORE_POSTGRES_DSN" appstore-20260901T080000Z.dump
```

복원 뒤에는 같은 버전의 이미지와 원래 **ENCRYPTION_KEY** 로 기동해 `/health/ready` 와 `/api/version` 을 확인하고, 앱 수·사용자·역할·감사 로그 기간과 OIDC·AI 연결 테스트 성공을 대조합니다. 자세한 절차는 [백업·복구 가이드](#)에 있습니다.

5.4. 업그레이드

마이그레이션은 기동 시 자동 적용됩니다. 그래서 **새 이미지를 올리기 전에 복구 가능한 백업을 확보하는 것이 가장 중요합니다.**

1. 현재 `/api/version`, 이미지 태그, 백업 시각을 기록합니다.
2. 위 5.3의 dump를 뜨고 `pg_restore --list` 로 읽히는지 확인합니다.
3. 새 archive의 SHA-256을 대조하고 `docker load` 후 label과 non-root 사용자를 확인합니다.
4. 교체합니다.

```
APPSTORE_VERSION=v2.11.4 docker compose up -d --no-build
docker compose ps
docker compose logs --tail 200 appstore
```

5. 새 이미지가 기동하면서 그 릴리스에 실린 서비스 가이드를 slug가 같은 앱에 자동 첨부합니다. 로그의 `bundled guides synced` 줄에서 `failed` 가 0인지 확인합니다.
6. `/health/live`, `/health/ready`, `/api/version` 이 새 버전인지 확인하고, 익명 탐색 → 로그인 → `/admin/users` 새로고침 → 앱 등록·검토 → 키 발급·회전·폐기 → AI 스트리밍 순으로 실제 화면을 확인합니다.

여러 인스턴스를 운영한다면 한 대만 먼저 올려 마이그레이션을 끝낸 뒤 나머지를 교체합니다. 같은 데이터베이스를 두 버전이 동시에 마이그레이션하지 않게 하려는 것입니다.

5.5. 롤백

자동 down 마이그레이션은 실행되지 않습니다. 새 마이그레이션이 이전 바이너리와 호환되지 않으면 이미지만 되돌려서는 안 되고 백업도 함께 복원해야 합니다.

이전 스키마와 호환되는 경우:

```
docker rm -f appstore
docker rename appstore-v2.11.3-stopped appstore
docker start appstore
curl --fail http://127.0.0.1:8080/health/ready
```

호환되지 않는 경우: 트래픽을 차단하고 → 변경 창 직전 dump를 별도 DB에 복원하고 → 원래 `ENCRYPTION_KEY` 와 이전 이미지를 연결하고 → `health`·로그인·핵심 화면을 확인한 뒤 트래픽을 복구합니다. 롤백 원인과 유실 가능 구간을 운영 기록에 남깁니다. 자세한 절차는 [업그레이드·롤백 가이드](#)에 있습니다.

6. 장애 대응

증상	확인할 곳	조치
컨테이너가 뜨자마자 죽는다. 로그에 missing required environment variables: ...	docker logs appstore 첫 줄	목록에 적힌 변수를 env 파일에 채웁니다
로그에 BOOTSTRAP_ADMIN_PASSWORD must be at least 12 characters	같은 곳	12자 이상으로 바꾸고 다시 기동합니다
로그에 initialize database (migrate): ...	PostgreSQL 도달 가능성, DSN, 계정 권한	DSN의 host·port·sslmode와 스키마 변경 권한을 확인합니다
로그에 ENCRYPTION_KEY does not match the initialized database	env 파일의 키	이 DB를 처음 초기화한 키를 써야 합니다. 키를 잃었다면 secret 재입력 계획을 세운 뒤 새 DB로 시작합니다
/health/live 는 200인데 /health/ready 가 503 { "status": "not_ready" }	PostgreSQL, 네트워크	DB가 죽었거나 연결이 끊겼습니다. 2초 안에 ping이 되어야 합니다
사용자에게 500과 요청 ID가 보인다	docker logs 에서 request_id 로 검색	panic recovered 면 stack 을, 아니면 해당 http request 의 status 와 path 를 봅니다
SSO 로그인에 안 된다	관리자 → 인증·SSO → 연결 테스트	3.3의 실패 메시지 표를 따릅니다
자동 로그인을 켜는데 로그인 화면이 뜬다	같은 브라우저 탭에서 Keycloak에 로그인돼 있는지, 주소에 sso=none 이 붙어 있는지 (로그인 화면에 "자동으로 로그인하지 않았습니다" 안내가 떠 있으면 그 경우입니다)	탭 세션당 한 번만 시도합니다. 새 탭에서 열거나 Keycloak에 먼저 로그인한 뒤 다시 엽니다. 로그아웃 직후에는 의도적으로 시도하지 않습니다
자동 로그인을 켜 뒤 화면이 깜빡이며 반복된다	Keycloak의 Valid Redirect URIs, docker logs 의 /api/v1/auth/oidc/callback 줄	정상이라면 일어나지 않습니다. 콜백이 /login?sso=none 으로 302를 돌려주는지 확인하고, 아니면 자동 로그인을 끄고 연결 테스트부터 다시 합니다
로그인은 되는데 화면이 403	사용자 → 역할, 인증·SSO → Role Mapping	외부 역할 값과 Role Claim Path가 실제 token과 맞는지 확인합니다
익명 사용자가 아무 화면도 못 본다	시스템 설정 → 공개 모드	꺼져 있으면 익명 탐색이 차단됩니다
API 호출이 자동화 API가 비활성화되어 있습니다.	관리자 → REST API	API 사용을 켭니다

증상	확인할 곳	조치
API·MCP 호출이 429 요청이 너무 많습니다.	REST API / MCP의 분당 요청 제한	한도를 올리거나 클라이언트가 Retry-After 초만큼 기다리게 합니다. reverse proxy 뒤에서는 제한이 프록시 IP 기준으로 잡힙니다
로고 업로드가 이미지는 1MB 이하여야 합니다.	파일 크기	1MB 이하로 줄입니다. 서버는 본문을 다 받기 전에 거절합니다
로고 주소 가져오기가 이미지 주소가 HTTP ...를 반환했습니다.	그 주소가 컨테이너에서 열리는지	폐쇄망에서 도달 가능한 주소인지, 인증이 필요한 주소는 아닌지 확인합니다
검토 대기 목록이 비어 있다	관리자 → 승인 워크플로	Workflow가 활성화된 경우에만 등록 건이 이 목록에 나타납니다.
팀장이 팀장 검토에는 사용자 팀 정보가 필요합니다. 를 받는다	사용자 → 담당팀	SSO에서 팀 정보가 오지 않았습니다. 사용자 정보의 팀을 채웁니다
AI 응답이 사용 가능한 AI Provider가 없습니다.	관리자 → AI 공급자	Provider와 모델을 등록하고 활성화합니다
추적을 켜는데 수집이 들어오지 않는다	관리자 → 방문 추적 → 정책이 차단한 출처	출처가 보이면 허용에 추가합니다. 비어 있으면 브라우저 콘솔의 Refused to load 줄과 스크립트의 <script>에 nonce 가 붙었는지 확인합니다. Momento는 같은 오리진 프록시를 켜는 편이 가장 확실합니다
/momento/* 가 404	관리자 → 방문 추적	추적 사용·provider Momento·같은 오리진 프록시가 모두 켜져 있어야 열립니다
/momento/* 가 502	docker logs 에서 momento proxy failed	컨테이너가 Momento 수집기 주소에 15초 안에 달아야 합니다

7. 보안

7.1. 기본값 중 바꿔야 하는 것

항목	기본값	운영에서
BOOTSTRAP_ADMIN_PASSWORD	<code>.env.example</code> 의 placeholder	길고 무작위한 값으로. vault에 보관하고 일상 계정으로 공유하지 않습니다
OIDC	꺼짐	조직 SSO를 붙이고 SSO Super Admin을 따로 지정합니다
자동 로그인 (Silent SSO)	꺼짐	모든 방문자에게 Keycloak 계정이 있는 사내 설치에서만 겁니다
서비스 접속 URL	빈 값	실제 HTTPS origin으로 채웁니다. redirect URL이 여기서 만들어집니다
API·MCP 익명 조회	컴	조직 정책에 따라 끄거나 최소 조회로 둡니다
공개 모드	컴	카탈로그를 사내에만 보여야 한다면 끕니다
API 120 / MCP 60 req·min	그대로	실제 사용량에 맞춰 조정합니다
방문 추적	꺼짐	켜야 한다면 Momento + 같은 오리진 프록시로. 외부 provider는 방문 데이터가 밖으로 나갑니다

7.2. 노출 범위

컨테이너는 평문 HTTP 8080만 listen합니다. 8080을 외부에 직접 열지 말고 reverse proxy가 127.0.0.1:8080에 붙게 하세요(`-p 127.0.0.1:8080:8080`). PostgreSQL은 서비스 네트워크 안에만 두고 외부에 열지 않습니다.

서비스가 스스로 붙이는 응답 헤더: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy (camera·microphone·geolocation·payment 모두 차단)`, `Cross-Origin-Opener-Policy: same-origin`, 그리고 `default-src 'self'` 기반 CSP(`frame-ancestors 'none'`, `object-src 'none'`, `script-src 'self'`). 방문 추적을 켜면 페이지 응답의 `script-src`에 요청마다 다른 `'nonce-...'`와 provider 출처가 더해지고(4.10), `/api/*·/mcp·/momento/*`는 `default-src 'none'`을 받습니다. reverse proxy에서 이 헤더를 약하게 덮어쓰거나 캐시하지 마세요 — nonce는 응답마다 달라야 합니다.

7.3. 비밀값 취급

- OIDC Client Secret과 AI API Key는 `ENCRYPTION_KEY`로 인증 암호화되어 DB에 저장되고, 조회 API가 없습니다. 변경만 가능합니다.
- 개인 API·MCP 키는 prefix와 keyed hash만 저장됩니다. 원문은 생성 순간 한 번만 표시되고 서버도 복원할 수 없습니다.
- `.env` 파일, DSN, 비밀번호, API Key를 커밋하거나 이미지 build argument로 넘기지 마세요.
- 로그·파비콘·배너 문구는 로그인 전 화면에도 보입니다. 비공개 정보를 넣지 마세요.

7.4. 컨테이너

이미지는 `non-root( appstore:appstore , uid/gid 10001)`로 실행하고, 배포는 `--read-only --cap-drop ALL --security-opt no-new-privileges:true` 로 굳힙니다. 쓰기가 필요한 곳은 `64MB tmpfs /tmp` 하나뿐입니다.

7.5. 운영 전 점검

- ☐ Bootstrap 관리자와 SSO Super Admin이 서로 다른 승인된 계정이다.
- ☐ OIDC·AI secret이 재조회되지 않고 변경만 가능하다.
- ☐ 공개 API와 익명 MCP가 조직 정책대로 설정돼 있다.
- ☐ 서비스 URL, OIDC redirect와 logout URL이 운영 HTTPS origin만 허용한다.
- ☐ PostgreSQL 백업과 ENCRYPTION_KEY 복구 절차를 각각 시험했다.
- ☐ 관리자·일반 사용자·익명 세션으로 200 / 401 / 403 경계를 확인했다.
- ☐ 감사 로그에 로그인과 설정 변경이 남는 것을 확인했다.

더 읽을 것

- [사용자 가이드](#) — 화면 사용법, 자주 하는 작업, 오류 메시지
- [오프라인 설치](#) · [업그레이드와 롤백](#) · [백업과 복구](#) · [릴리스 Runbook](#)
- 서비스의 `/docs` 와 `/openapi.json` 에 REST API 계약이 있습니다.